Theory of Computation @ HNU CE (2026년 2학기)

kya

2026-11-02

차 례

HW03 TODO 4
1. CFG를 top-down PDA로 4
2. CFG를 bottom-up PDA로 6
3. CFG를 무한루프를 회피하는 bottom-up PDA로 구현 8
4. DCG로 작성한 문법규칙이 문자열을 수락하며 문법분석나무도 생성 9
5. Top-down PDA로 작성한 Prolog 코드에서도 문법분석나무 생성 10
1. Prolog 소개 11
1.1. 가족 관계 예시 11
1.2. 리스트 기초, 출력을 위한 술어, 연결자(connective) 우선순위 12
1.3. 리스트 연결 예시 14
1.4. 사실(fact), 규칙(rule), 절(clause), 술어(predicate) 16
1.5. 항(term)과 일치화(unification) 18
1.5.1. 항(term) 18
1.5.2. 일치화(unification) 19
1.6. `maplist`와 `sort` 21
1.7. 부분집합(subset)과 멱집합(powerset) 24
1.8. 확정절 문법(Definite Clause Grammar) 26
주교재 0 Introduction 30
주교재 0.1 오토마타, 계산가능성, 복잡도 30
Complexity Theory 복잡도 이론 31
Computability Theory 계산가능성 이론 31
Automata Theory 오토마타 이론 31
주교재 0.2 수학적 표기법과 용어 32
집합 set 32
함수와 관계 functions and relations 34
관계의 표기법 36
그래프 Graph 37
문자열과 언어 38
주교재 1 정규언어 Regular Languages 39
주교재 1.1 유한 오토마타 Finite Automata 39
주교재 1.2 비결정성 Non-determinism 50
NFA와 DFA는 동치/등가(equivalent) 53
주교재 1.3 정규식 Regular Expressions 63
주교재 1.3 정규언어가 아닌 언어 66

주교재 2 Context-Free Languages 문맥자유언어 67
주교재 3 처치–튜링 명제/논제/테제/가설 81
3.1 Turing Machines 82

HW03 TODO

- 이름:
- 학번:

1. CFG를 top-down PDA로

(1) $S \to 0S1$ (2) $S \to x$ 두 규칙의 CFG를 top-down 방식의 PDA로 작성하고 테스트.

tc26hw03a.pl

```
state(q_start). state(q_s2l).                    % Q (states)
state(q_loop). state(q_1__1). state(q_1_S1).   state(q_accept).
sigma(0).   sigma(1).   sigma(x).                % Σ (terminal symbols)
gamma('$').  gamma('S').  gamma(X) :- sigma(X). % Γ (stack symbols)
% 수식으로 된 수학적 정의보다는 PDA 전이도(그림)의 구조와 맞게 delta 작성
delta(q_start, [],    []-['$'], q_s2l).
delta(q_s2l,   [],    []-['S'], q_loop).
delta(q_loop,  [], ['S']-[ 1 ], q_1__1).  % rule (1) __1
delta(q_loop,  [],    []-['S'], q_1_S1).  % rule (1) _S1
delta(q_loop,  [],    []-[ 0 ], q_loop).  % rule (1) 0S1, back to q_loop
delta(q_loop,  [], ['S']-[ x ], q_loop).  % rule (2)
delta(q_loop, [A], [ A ]-[],    q_loop) :- sigma(A).  % match terminal
delta(q_loop,  [], ['$']-[],    q_accept).
% delta(Q1, A, S1-S2, Q2) 형식 검사 위한 helper predicates
sigmaE([A]) :- sigma(A).  sigmaE([]).
gammaE([X]) :- gamma(X).  gammaE([]).
% every Q1 and Q2 is a state
?- findall(Q1, delta(Q1,_,_,_), QS), maplist(state, QS).
?- findall(Q2, delta(_,_,_,Q2), QS), maplist(state, QS).
% every X are either a singleton alphabet symbol list or the empty list
?- findall(A,  delta(_,A,_,_), AS), maplist(sigmaE, AS).
% every S1 and S2 is either a singleton stack symbol list or the empty list
?- findall(S1, delta(_,_,S1-_,_), SS), maplist(gammaE, SS).
?- findall(S2, delta(_,_,_-S2,_), SS), maplist(gammaE, SS).

% delta를 0회 이상 여러 번에 걸쳐 계속 진행시키는 과정 (Q1,W1,S1)->...->(Q2,W2,S2)
steps(Q1, W1, S1, Q2, W2, S2) :- true.  % TODO 여기 true 대신 내용 작성.
                                        % 필요하면 여러 규칙으로 나눠 작성

runPDA(Input) :- steps(q_start, Input, ['$'], q_accept, [], _).

% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 1
% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 2
% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 3
```

```
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/tc26hw03a.pl:27:
O:
E: Warning:    Singleton variables: [Q1,W1,S1,Q2,W2,S2]
```

2. CFG를 bottom-up PDA로

(1) $S \to 0S1$ (2) $S \to x$ 두 규칙의 CFG를 bottom-up 방식의 PDA로 작성하고 테스트.

tc26hw03b.pl

```
state(q_start).
state(q_loop). state(q_l2a).
state(q_accept).
% TODO 위 4개 외에 다른 필요한 state들 여기에 추가       % Q (states)
sigma(0).   sigma(1).   sigma(x).                        % Σ (terminal symbols)
gamma('$').    gamma('S').    gamma(X) :- sigma(X). % Γ (stack symbols)
% 수식으로 된 수학적 정의보다는 PDA 전이도(그림)의 구조와 맞게 delta 작성
delta(q_start, [],    []-['$'], q_loop).
delta(q_loop, [A],    []-[ A ], q_loop) :- sigma(A).  % shift terminal
% delta(...)  % TODO 적절한 reduce 관련 전이들 작성
% delta(...)  % 그 외에 다른 필요한 전이가 있다면 작성
delta(q_loop,  [], ['S']-[],    q_l2a).
delta(q_l2a,   [], ['$']-[],    q_accept).

% delta(Q1, A, S1-S2, Q2) 형식 검사 위한 helper predicates
sigmaE([A]) :- sigma(A).  sigmaE([]).
gammaE([X]) :- gamma(X).  gammaE([]).
% every Q1 and Q2 is a state
?- findall(Q1, delta(Q1,_,_,_), QS), maplist(state, QS).
?- findall(Q2, delta(_,_,_,Q2), QS), maplist(state, QS).
% every X are either a singleton alphabet symbol list or the empty list
?- findall(A,  delta(_,A,_,_), AS), maplist(sigmaE, AS).
% every S1 and S2 is either a singleton stack symbol list or the empty list
?- findall(S1, delta(_,_,S1-_,_), SS), maplist(gammaE, SS).
?- findall(S2, delta(_,_,_-S2,_), SS), maplist(gammaE, SS).

% TODO 수동으로 몇 단계에 걸쳐 delta를 연쇄적으로 이어붙인 전이과정 확인용 예시 작성
% ?- delta(...),...,delta(...),...,format(...). % 이렇게 수업노트에 나온 것처럼

% delta를 0회 이상 여러 번에 걸쳐 계속 진행시키는 과정 (Q1,W1,S1)->...->(Q2,W2,S2)
steps(Q1, W1, S1, Q2, W2, S2) :- true.  % TODO 여기 true 대신 내용 작성.
                                        % 필요하면 여러 규칙으로 나눠 작성

runPDA(Input) :- steps(q_start, Input, ['$'], q_accept, [], _).

% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 1
% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 2
% ?- runPDA(...), ... . % TODO 적절한 방식으로 runPDA 테스트 및 결과 출력 3
```

```
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/tc26hw03b.pl:31:
O:
E: Warning:    Singleton variables: [Q1,W1,S1,Q2,W2,S2]
```

3. CFG를 무한루프를 회피하는 bottom-up PDA로 구현

CFG를 bottom-up 방식의 PDA로 곧이곧대로 Prolog로 옮겨 작성하면 무한루프로 인해 스택 오버플로가 발행할 수 있는 경우의 CFG를 제시하고, 이를 Prolog 실행시 회피하기 방법을 추가하여 작성하고 테스트하라. 최대한 2번의 문제에 제시된 코드 틀을 되도록 그대로 따르면서 필요한 부분만 코드의 구조에 변경을 가하여 작성하라.

무한루프를 실제로 발생시켜 스택 오버플로를 출력할 필요는 없고, 회피하는 구현만 작성하면 된다.

단, 제시한 CFG에서 왜 무한루프가 발생하게 되는지 여기에 일반 텍스트 및 수식으로 설명하라.

1. 구현할 CFG 규칙들을 아래에 작성 (4튜플 형태로 작성할 필요 없고 간단히 문법 들만 작성)

   ```
   S -> ... TODO
   . -> ... TODO
   ```
2. 무한루프로 진행하는 과정을 간단히 요약한 예시를 들면서, 너무 길지 않게 무한루프 발생하는 이유를 간결하게 핵심만 설명

   TODO 여기에 설명 작성

```
% TODO 여기에 코드 작성
```
tc26hw03c.pl

```
0:
```

4. DCG로 작성한 문법규칙이 문자열을 수락하며 문법분석나무도 생성

이 문서의 “Prolog 소개”의 확정절 문법을 다루는 절에서 DCGExample3.pl을 시작으로 변경하여 작성하라.

이 문제는 Prolog의 항(term)이 트리 구조를 나타낸다는 것을 분명히 이해하고 활용할 수 있어야 한다. 이 개념이 이해가 안간다면 강의노트 Prolog 소개의 관련 부분도 다시 읽어보고 LLM 챗봇 서비스를 활용하여 Prolog의 항(term)이 트리 구조를 나타낸다는 원리에 대해 질문해 보며 먼저 그 개념을 확실히 하고 넘어가야 한다.

```
1 % TODO 여기에 코드 작성
```

tc26hw03d.pl

```
0:
```

5. Top-down PDA로 작성한 Prolog 코드에서도 문법분석나무 생성

앞의 문제의 CFG를 이번에는 top-down 방식 또는 bottom-up 방식의 PDA 중 하나로 구현하면서 문자열을 인식하는 과정에서 문법분석나무도 함께 생성하도록 작성하라. PDA 실행 과정에 대한 기록을 로그로 남겨서 문자열을 수락하고 난 다음 별도의 단계에서 문법분석나무를 생성하지 말고 PDA 실행 과정에서 같이 점차적으로 생성되도록 하되 Prolog의 논리 변수를 적절히 활용하라. 앞의 4번 문제의 PDA버전을 작성하라는 문제이다.

tc26hw03e.pl

```
% TODO 여기에 코드 작성
```

```
0:
```

1. Prolog 소개

Prolog 코드에서 `%`로 시작해 줄 끝까지 부분은 주석(comment)으로 취급됨.

1.1. 가족 관계 예시

family.pl

```
% 1항 관계 male    원소나열법 표현
male(bob).      male(john).     % 원래 한줄에 하나씩 작성을 권장
% 1항 관계 female  원소나열법 표현
female(alice).  female(carol).  % 원래 한줄에 하나씩 작성을 권장
% 2항 관계 child.  원소나열법 표현
child(bob, alice).  child(carol, alice).
child(bob, john).   child(carol, john).
% 2항 관계 son     조건제시법 표현
son(X, Y) :- child(X, Y), male(X).

?- findall( (X,Y), son(X, Y) , Bag ),
   forall( member((X,Y), Bag),
           format("son: X = ~w, Y = ~w~n", [X, Y]) ).
```

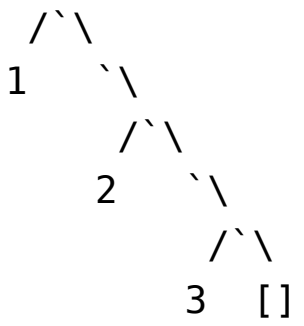
```
0: son: X = bob, Y = alice
0: son: X = bob, Y = john
0:
```

1.2. 리스트 기초, 출력을 위한 술어, 연결자(connective) 우선순위

Prolog에서 리스트(list)는

- 길이 0인 빈 리스트는 `[]`로 표현
- `[1,2,3]`처럼 쉼표(`,`)로 구분하여 나열한 원소를 대괄호(`[`와 `]`)로 감싸서 표현
- `[H|T]`처럼 하나의 원소(`H`, 머리)와 나머지 리스트(`T`, 꼬리)로 나누어 표현

```
[ 1,  2,  3       ] =
[ 1| [2,  3     ] ] =
[ 1| [2| [3   ] ] ] =
[ 1| [2| [3|[]] ] ]
```



실제로 `write` 술어(predicate)로 출력해 보면 똑같다.

listbasic.pl

```
?- L = [ 1,  2,  3       ], write(L), nl.
?- L = [ 1| [2,  3     ] ], write(L), nl.
?- L = [ 1| [2| [3   ] ] ], write(L), nl.
?- L = [ 1| [2| [3|[]] ] ], write(L), nl.
```

```
0: [1,2,3]
0: [1,2,3]
0: [1,2,3]
0: [1,2,3]
```

Prolog에서는 `조건 -> 참인_경우 ; 거짓인_경우` 형태로 if-then-else를 표현 가능

```
6  ?- [1,2,3] = [1,[2,3]]  -> (write(true), nl) ; (write(false), nl).
7  ?- [1,2,3] = [1|[2,3]]  -> (write(true), nl) ; (write(false), nl).
```

listbasic.pl

```
0: false
0: true
```

and 연결자(`,`)의 우선순위가 if 연결자(`->`)와 or 연결자(`;`)보다 높으므로 괄호 생략 가능

```
9  ?- [1,2,3] = [1,[2,3]]  ->  write(true), nl  ;  write(false), nl .
10 ?- [1,2,3] = [1|[2,3]]  ->  write(true), nl  ;  write(false), nl .
```

listbasic.pl

```
0: false
0: true
```

하나의 파일 `listbasic.pl`을 이렇게 여러 부분의 코드 조각으로 나누어 작성하는 것도 가능하다.

좀 더 복잡한 형식의 출력은 C의 printf와 비슷한 `format` 술어를 활용

```
12 ?- L1 = [1], L2 = [2,3], format("L1 = ~w, L2 = ~w", [L1,L2]).
```

listbasic.pl

```
0: L1 = [1], L2 = [2,3]
0:
```

1.3. 리스트 연결 예시

`app(L1,L2,L)`은 두 리스트 `L1`과 `L2`를 이어붙이면 `L`이 되는 3항 관계

listapp.pl

```
app([], L, L).                              % 빈 리스트와 L을 연결하면 L
app([H|L1], L2, [H|L]) :- app(L1, L2, L).   % 길이 1이상의 리스트와 L2 연결하는 재귀적 규칙

?- app([1], [2,3], L),          % 관계를 만족하는 L을 탐색
   format("L = ~w~n", [L]).     % L을 적절한 형식으로 출력
```

```
0: L = [1,2,3]
```

원래 `app`을 정의할 때 생각했던 것과 반대 방향으로도 활용 가능!

- `L1`과 `L2`를 연결해 `L`이 되는 관계를 생각하고 정의했는데 …
- 반대로 `L`을 `L1`과 `L2`로 나누는 방법도 탐색 가능!!!

(참고로, 이런 반대 방향 탐색이 모든 정의에 대해 항상 잘 되는 것은 아님)

listapp.pl

```
?- app(L1, L2, [1,2,3]),        % 관계를 만족하는 1가지 경우를 찾아서
   format("L1 = ~w, L2 = ~w~n", [L1, L2]). % 적절한 형식으로 출력
```

```
0: L1 = [], L2 = [1,2,3]
```

길이 0인 `L1`인 당연한 경우만 탐색되어 실망했다면 … 다음 페이지를 보라

`app(L1,L2,[1,2,3])`를 만족하는 모든 경우를 탐색하려면 `findall`을 활용해 질의(query)

```
10 ?- findall( (L1,L2), app(L1, L2, [1,2,3]), Bag ),  % 모든 경우의 집합 Bag
11    forall( member((L1,L2), Bag),  % 모든 경우를 순회하며 Bag의 원소 [L1,L2]를 하나씩
12            format("L1 = ~w, L2 = ~w~n", [L1, L2]) ).        % 적절한 형식으로 출력
```

listapp.pl

```
0: L1 = [], L2 = [1,2,3]
0: L1 = [1], L2 = [2,3]
0: L1 = [1,2], L2 = [3]
0: L1 = [1,2,3], L2 = []
```

최대 몇 개까지만 찾으려면 `findsols`를 활용해 질의(query)

```
14 ?- findnsols( 2,  % 최대 2개까지만 만족하는 해를 찾기
15               (L1,L2), app(L1, L2, [1,2,3]), Bag ),
16    forall( member((L1,L2), Bag),
17            format("L1 = ~w, L2 = ~w~n", [L1, L2]) ).
```

listapp.pl

```
0: L1 = [], L2 = [1,2,3]
0: L1 = [1], L2 = [2,3]
0:
```

1.4. 사실(fact), 규칙(rule), 절(clause), 술어(predicate)

술어(predicate)

- 일반적으로 관계(relation)를 정의함
- 하나 이상의 절(clause)로 구성됨

절(clause)은 다음 두 형태 중 하나

- 사실(fact)
  - 머리(head) 부분 다음에 `:-` 없이 `.`으로 끝나는 형태
  - 머리 부분이 나타내는 성질이 아무런 조건 없이 항상 참
- 규칙(rule)
  - 머리(head) 부분 다음 `:-`와 몸체(body) 부분이 오고 `.`로 끝나는 형태
  - 머리 부분이 나타내는 성질이 특정 조건(즉, 몸체가 나타내는 성질)이 성립할 때만 참

```
%% 사실들만으로 정의된 술어 male
male(bob).
male(john).
%% 규칙 하나로만 정의된 술어 father
father(X) :- child(C,X), male(X).    % 자녀가 있고 남자인 조건이 성립해야 아버지로 인정

%% 사실과 규칙 각각 하나씩 2개의 절로 정의된 술어 is_list
is_list([]).                  % 사실(fact): []는 무조건 리스트로 인정
is_list([H|T]) :- is_list(T). % 규칙(rule): [H|T]는 is_list(T)가 성립해야 리스트로 인정
```

사실(fact)을 똑같은 의미의 규칙(rule) 형태로 바꿔 쓰려면
머리 다음에 바로 `.`으로 끝내는 대신 뒤에 `:- true.`라고 쓰면 됨.

```
male(bob)  :- true.  % male(bob). 과 똑같은 의미
male(john) :- true.  % male(john). 과 똑같은 의미
```

왜냐하면 `true`는 무조건 참으로 Prolog에서 미리 정의된 특별한 술어이기 때문.

Prolog의 규칙 형태의 절 C `:-` P_1 `,` P_2 `,` $\cdots$ `,` P_n `.` 는

추론규칙 $\dfrac{P_1 \quad P_2 \quad \cdots \quad P_n}{C}$ 혹은 논리식 $P_1 \wedge P_2 \wedge \ldots \wedge P_n \Longrightarrow C$ 에 해당한다.

(참고로, 논리식에서는 $\Longrightarrow$ 대신 $\longrightarrow$나 $\supset$기호를 쓰기도 함)

1.5. 항(term)과 일치화(unification)

1.5.1. 항(term)

Prolog에서 술어(predicate)가 뭔가 실행되어 움직이는 동적인 무언가라면,
항(term)은 정적인 데이터에 해당한다. 항(term)은 실행되는 것이 아님!!!

항(term)은 다음 셋 중 하나의 형태

- 변수(variable): `X`, `Y1` 처럼 대문자로 시작. functor 위치에는 올 수 없음.
- 상수(constant)
  - atom: `a`, `b1`처럼 소문자로 시작하거나 `'Hello World'` 처럼 따옴표로 둘러싸인 것
  - number: `123`같은 정수나 `3.14`같은 실수
- compound term / structure: `f(t1, ...)`
  - 여기서 `f`는 함수자(functor) / 함수 기호(function symbol)
  - 여기서 `t1`, … 등등은 항(term)

functor의 항수(arity)

- `f(t1,t2)`에서 `f`의 항수는 2
- `f(t1,t2,t3)`에서 `f`의 항수는 3

프로그램 코드에서는 같은 `f`지만 Prolog 내부적으로 전자는 `f/2`, 후자는 `f/3`와 같이 처리해 위의 두 `f`를 서로 다른 functor로 확실히 구분함!!! (`g`, `h`, `f1`, `f4`처럼 이름이 다른 경우나 마찬가지)

1.5.2. 일치화(unification)

일치화(unification)란 두 항(term)을 서로 동일하게 만들 수 있는지 판단하는 과정이다. 그 과정에서 필요하다면 변수를 항으로 치환(substitute)함으로써 (혹은, 변수에 항을 대입함으로써) 두 항을 동일하게 만든다.

변수가 없는 경우에는 단순히 판단하기만 하면 됨

unification.pl

```
?- a = 3 -> write(true), nl ; write(false), nl.
?- a = a -> write(true), nl ; write(false), nl.
```

```
0: false
0: true
```

unification.pl

```
?- f(1,3) = g(1,3) -> write(true), nl ; write(false), nl.
?- f(1,a) = f(1,3) -> write(true), nl ; write(false), nl.
?- f(1,3) = f(1,3) -> write(true), nl ; write(false), nl.
```

```
0: false
0: false
0: true
```

변수에 무엇을 대입해도 일치화할 수 없는 경우들

unification.pl

```
8  ?- f(1,a) = f(X,3) -> format('success: X = ~w~n', [X])
9                      ; format("failure~n").
10 ?- f(1,Y) = g(X,3) -> format('success: X = ~w, Y = ~w~n', [X,Y])
11                     ; format("failure~n").
```

```
0: failure
0: failure
```

변수를 적절히 치환하여 일치화에 성공하는 경우들

unification.pl

```
13 ?- f(1,3) = f(X,3) -> format('success: X = ~w~n', [X])
14                     ; format("failure~n").
15 ?- f(1,Y) = f(X,3) -> format('success: X = ~w, Y = ~w~n', [X,Y])
16                     ; format("failure~n").
```

```
0: success: X = 1
0: success: X = 1, Y = 3
0:
```

살펴본 바와 같이 Prolog는 항(term)에 대한 일치화(unification) 알고리즘을 자체적으로 내장하고 있으므로, Prolog에서 제공하는 두 항의 일치화 기능을 위해 제공하는 특수한 2항 술어(predicate)인 `=`를 사용하기만 하면 됨.

1.6. `maplist`와 `sort`

예를 들어 `maplist(f(A1,A2,...), [X1,X2,X3,...], [Y1,Y2,Y3,...])`는
`f(A1,A2,...,X1,Y1), f(A1,A2,...,X2,Y2), f(A1,A2,...,X3,Y3), ...`와 같다.

즉, 첫번째에 제공된 항(term), 즉 `f(A1,A2,...)`의 인자들 끝에
두번째 이후로 나타나는 리스트들에서 있는 같은 위치에 있는 원소들을 추가한 다음
함수 기호 `f`를를 술어로 해석하여 모두 동시에 만족되는지 알아보도록 실행한다.

maplist1.pl

```
even(N) :- 0 is N mod 2.

% maplist over even/1
?- ( maplist(even, [0,2,4,6,8]) -> X='all'; X='not all' ),
   format('~w numbers are even~n',[X]).
?- ( maplist(even, [0,2,4,7,8]) -> X='all'; X='not all' ),
   format('~w numbers are even~n',[X]).
```

```
0: all numbers are even
0: not all numbers are even
```

이렇게 하나의 리스트에 대해서도 가능하고

maplist1.pl

```
9  ?- sort([2,4,1,3], Ys), format('sorted list: ~w~n', [Ys]).
10
11 % maplist over sort/2
12 ?- maplist(sort, [[2,1,3,4],[red,green,yellow],[99,88,77,55,66]], Ys),
13    format('many sorted lists: ~w~n',[Ys]).
14 % maplist over append/3 with its first argument aready provided
15 ?- maplist(append([1,2]), [[3],[4,5],[6,7,8]], Zs),
16    format('many lists with prefix [1,2]: ~w~n',[Zs]).
```

```
0: sorted list: [1,2,3,4]
0: many sorted lists: [[1,2,3,4],[green,red,yellow],[55,66,77,88,99]]
0: many lists with prefix [1,2]: [[1,2,3],[1,2,4,5],[1,2,6,7,8]]
```

이렇게 두 개의 리스트에 대해서도 가능하고 …

아래처럼 세 개나 네 개 등등도 됨

maplist1.pl

```
18 % maplist over append/3
19 ?- maplist(append, [[a1,a2],[b1,b2,b3],[c1]], [[3],[4,5],[2,3,4]], Zs),
20    format('many appended lists: ~w~n',[Zs]).
```

```
0: many appended lists: [[a1,a2,3],[b1,b2,b3,4,5],[c1,2,3,4]]
```

참고로 `sort`는 중복도 제거 (집합을 리스트로 표현할 때는 오히려 좋아!)
중복 제거 없이 중복된 원소의 개수를 유지하려면 `msort`

maplist1.pl

```
22 ?- sort([2,1,4,3,1,2,5,4,3], Ys), format('sort result: ~w~n',[Ys]).
23 ?- msort([2,1,4,3,1,2,5,4,3], Ys), format('msort result: ~w~n',[Ys]).
```

```
0: sort result: [1,2,3,4,5]
0: msort result: [1,1,2,2,3,3,4,4,5]
0:
```

1.7. 부분집합(subset)과 멱집합(powerset)

subset1.pl

```
% subset(A,S) when A is a subset of S
subset(    [],    []).
subset([X|Xs],[X|Ys]) :- subset(Xs,Ys).  % 첫 원소를 부분집합에 포함하는 경우
subset(   Xs, [_|Ys]) :- subset(Xs,Ys).  % 첫 원소를 부분집합에서 제외하는 경우

% powerset(S,PS) when the powerset of S is PS
powerset(S, PS) :- findall(X, subset(X,S), PS).

?- subset([2],[1,2]) -> format('subset test 1 success.~n')
                      ; format('subset test 1 failure.~n').
?- subset([3],[1,2]) -> format('subset test 2 success.~n')
                      ; format('subset test 2 failure.~n').

?- powerset([1,2,3], PS), format('All subsets: ~w~n',[PS]).
```

```
0: subset test 1 success.
0: subset test 2 failure.
0: All subsets: [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
0:
```

이렇게 findall을 술어의 구현으로 활용 가능
하지만 이렇게 특수한 기능을 활용하면 PS로부터 S를 역산할 수는 없음에 유의

subset2.pl

```
% subset(A,S) when A is a subset of S
subset(    [],    []).
subset([X|Xs],[X|Ys]) :- subset(Xs,Ys).  % 첫 원소를 부분집합에 포함하는 경우
subset(   Xs, [_|Ys]) :- subset(Xs,Ys).  % 첫 원소를 부분집합에서 제외하는 경우

% powerset(S,PS) when the powerset of S is PS
powerset(    [], [[]]).     % 공집합 {}의 멱집합은 { {} }
powerset([X|Xs], PS) :-
  powerset(Xs,PS0),          % PS0는 첫 원소를 제외 나머지 원소들에 대한 멱집합
  maplist(cons(X),PS0,PS1), % PS1 = { [X|Xs] | X in PS0 }
  append(PS0,PS1,PS).        % PS0과 PS1의 합집합이 PS

cons(X,Xs,[X|Xs]).

?- powerset([1,2,3], PS), format('All subsets: ~w~n',[PS]).
?- powerset(A, [[],[2],[1],[1,2]]), format('Orignal set: ~w~n',[A]).
```

```
0: All subsets: [[],[3],[2],[2,3],[1],[1,3],[1,2],[1,2,3]]
0: Orignal set: [1,2]
0:
```

그런데 어차피 멱집합으로부터 역산할 일은 실제로는 거의 없기 때문에
실용적으로는 어떻게 구현하는 보통은 상관없음

1.8. 확정절 문법(Definite Clause Grammar)

우리 교재의 CFG를 Prolog의 DCG로 옮기면 실행할 수 있다.

두 개의 문법 규칙 $A \longrightarrow aA$와 $A \longrightarrow b$를 DCG로 옮겨 실행해 보면 다음과 같다.

DCGexample1.pl

```
'A' --> [a], 'A'.
'A' --> [b].

?- L = [a],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [a,b],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [a,a,a,b],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [a,a,a,b,b],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
```

```
0: [a] 거부.
0: [a,b] 수락.
0: [a,a,a,b] 수락.
0: [a,a,a,b,b] 거부.
0:
```

주교재 2장에 처음 나오는 CFG 문법 규칙을 Prolog의 DCG로 옮겨서 실행해 보면 다음과 같다.

DCGexample2.pl

```
'A' --> [0], 'A', [1].
'A' --> 'B'.
'B' --> [x]. % 교재의 # 대신 x 로

?- L = [0,1],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [0,x,1],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [0,0,0,x,1,1,1],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
?- L = [0,0,0,x,1,1,1,1],
   ( phrase('A', L) -> format('~w 수락.~n',[L]); format('~w 거부.~n',[L]) ).
```

```
0: [0,1] 거부.
0: [0,x,1] 수락.
0: [0,0,0,x,1,1,1] 수락.
0: [0,0,0,x,1,1,1,1] 거부.
0:
```

하지만 교재의 문법 규칙들 중에는 그대로 옮기면 무한 재귀로 인해 스택 오버플로우가 발생하는 경우도 있다.

DCGexample3.pl

```
'S' --> [a], 'S', [b].
'S' --> 'S', 'S'.      % 이것 때문에 stack overflow 발생 가능
'S' --> [].

?- phrase('S', [a,b,a,b]).
% ?- phrase('S', [a,b,a,a,b,b]).
% ?- phrase('S', [a,b]).
% ?- phrase('S', [a,a,b]).
% ?- phrase('S', [a,a,b,b]).
```

```
E: ERROR: /home/kyagrd/github/hnu-pl/TC2026/_build/DCGexample3.pl:5:
E: ERROR:    Stack limit (1.0Gb) exceeded
E: ERROR:      Stack sizes: local: 1.0Gb, global: 48.8Mb, trail: 1Kb
E: ERROR:      Stack depth: 6,390,766, last-call: 0%, Choice points: 6,390,744
E: ERROR:      Probable infinite recursion (cycle):
E: ERROR:        [6,390,766] user:'S'([length:3], _12788516)
E: ERROR:        [6,390,765] user:'S'([length:3], _12788542)
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/DCGexample3.pl:5:
O:
E: Warning:    Goal (directive) failed: user:phrase('S',[a,b,a,b])
```

놀랍게도 여기서도 테이블링 마법이 통한다!!!
이게 왜 되는지 이해하려 노력하면 많은 깨달음이 있을 것이다.

DCGexample4.pl

```
:- table 'S'/2.   %% DCG의 hidden parameter 2개이므로
% :- table 'S'//0.  %% DCG 전용 테이블링 문법 rule_name//k 는 rule_name/k+2 에 해당
'S' --> [a], 'S', [b].
'S' --> 'S', 'S'.      % 이것 때문에 stack overflow 발생 가능
'S' --> [].

?- phrase('S', [a,b,a,b]).
?- phrase('S', [a,b,a,a,b,b]).
?- phrase('S', [a,b]).
?- phrase('S', [a,a,b]).
?- phrase('S', [a,a,b,b]).
```

```
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/DCGexample4.pl:10:
O:
E: Warning:    Goal (directive) failed: user:phrase('S',[a,a,b])
```

주교재 0 Introduction
주교재에서 다루는 영역에 대한 개괄적 소개.

주교재에서 설명하면서 필요한 수학적 개념들 미리 정리.

주교재 0.1 오토마타, 계산가능성, 복잡도
계산이론(theory of computation)이 다루는 세 가지 영역
- 오토마톤 / 상태 기계: Automaton / State Machine
- 계산가능성: Computability - 어떤 문제가 계산으로 정의/해결 가능한지
- (계산)복잡도: (Computational) Complexity - 계산하기 위해 필요한 자원 소모 정도

참고로 오토마타(Automata)는 오토마톤(Automaton)의 복수형.
근데 한국어에서는 귀찮으니 그냥 1개도 오토마타라고 부르는 경우가 많음.
영어로 쓸 때는 주의해야 함.

계산이론의 이런 모든 분야를 관통하는 핵심 질문:
“What are the fundamental capabilities and limits of computation?”
“계산이라는 절차가 근본적인 역량과 한계는 무엇인가?”

즉, 계산이라는 것으로 어디까지는 할 수 있고 어떤 범위에서는 계산이라는 것이 불가능한지 탐구하는 것이 계산이론의 핵심이다.

(계산할 수 있을 거 같은데 … 안되는 것도 있다는 이야기)

Complexity Theory 복잡도 이론
계산의 난이도를 다루는 이 분야의 핵심 질문:
“What makes some problems computationally hard and others easy?”
“어떤 문제들은 계산적으로 어렵고 다른 문제들은 쉬운 것은 무엇 때문인가?”

알고리즘 과목에서 많이 다루는 개념. (근데 일단 계산이 가능한 것들 중에서 …)

Computability Theory 계산가능성 이론
이 분야의 선구자들: 쿠르트 괴델, 앨런 튜링, 알론조 처치

이 이론을 전개하기 위해서는 계산이 무엇인지 정의해야 함

계산의 모델(model)이라는 것을 정의하는 방식으로 계산이 무엇인지 정의함

Automata Theory 오토마타 이론
계산이 무엇인지 정의하는 한 방식으로 활용됨. 계산의 모델을 명확히 제시해야 계산이 가능한지, 가능하다면 그 계산이 얼마나 어려운지(필요 시간, 공간 등 자원) 판단 가능.

여러 모델들이 다른 분야에서도 응용됨
- 유한 오토마타 / 유한 상태 기계: Finite Automata / Finite State Machine
  - 텍스트 처리, 컴파일러에서 어휘 분석, 등등
- 푸시다운 오토마타 / 스택 오토마타: Pushdown Automata / Stack Automata
  - 언어의 문법 분석, 등등

주교재 0.2 수학적 표기법과 용어

집합 set
집합(set)은 원소/멤버(element/member)들의 모임.

집합의 원소나열법 표기: $S = \{7, 12, 57\}$

원소와 집합의 포함 관계 표현: $7 \in S$, $8 \notin S$

원소나열법과 생략 기호를 활용한 표기:
- $\mathcal{N} = \{0, 1, 2, ...\}$ (자연수의 집합)
- $\mathcal{Z} = \{..., -2, -1, 0, 1, 2, ...\}$ (정수의 집합)

집합의 조건제시법 표기(set-builder notation): $S = \{m^2 \mid m \in \mathcal{N}\}$ (자연수의 집합)

공집합: $\varnothing = \{\}$ (원소가 하나도 없는 빈 집합)

원소가 딱 1개인 "한원소 집합"(singleton set). 예: $\{7\}$ (원소가 7 하나뿐인 집합)

원소가 딱 2개인 집합을 "순서 없는 쌍"(unordered pair) 또는 "집합쌍"이라고 함.
예: $\{7, 12\} = \{12, 7\}$ (원소가 7과 12뿐인 집합)

순서쌍(ordered pair, pair, 2-tuple). 예: $(7, 12) \neq (12, 7)$ (순서가 중요)
일반적으로 k-tuple. 예: $(7, 12, 5)$ (3-tuple)

집합의 연산

- 교집합(intersection): $A \cap B$
- 합집합(union): $A \cup B$
- 여집합(complement): $\overline{A} = \{x \mid x \in U, x \notin A\}$ 전체집합(U)이 정의되어 있어야 함
- 곱집합(cartesian product): $A \times B = \{(x, y) \mid x \in A, y \in B\}$
- 멱집합(power set): $\mathcal{P}(A) = 2^A = \{B \mid B \subseteq A\}$

집합 3개를 연속해서 곱집합(cartesian product)하면 3-tuple들의 집합이 만들어짐: $A \times B \times C = \{(x, y, z) \mid x \in A, y \in B, z \in C\}$
이런 식으로 정의되는 3-tuple에서는 $(a, b, c) = ((a, b), c) = (a, (b, c))$를 동치로 취급함. 즉, $(A \times B) \times C = A \times (B \times C) = A \times B \times C$ 라는 말. 하지만, 경우에 따라서는 이걸 구분해서 다루기도 하므로 주의가 필요. 맥락을 잘 살펴야.

벤다이어그램(Venn diagram)은 주교재 그림 참고

집합의 거듭제곱 표기
- $A^1 = A$
- $A^2 = A \times A$
- $A^3 = A \times A \times A$
- $\cdots$

일반적으로 $A^{k+1} = A \times A^k$

그런데 A^0 은 무엇이 되어야 할까? 일단, 공집합($\varnothing$)은 안됨!

유한집합의 경우 $A \times B$의 크기(원소의 개수)는 각각의 크기(원소의 개수)의 곱.
한쪽 크기가 0이면 곱집합의 크기도 0이 됨. 따라서, A^0은 한원소 집합이어야 함.
근데, 그 한 원소를 뭘로 하지?

$A^0 = \{()\}$ (한원소 집합, 그 한 원소 $()$는 0-tuple)

$A = \{a_1, a_2, ...\} = \{(a_1), (a_2), ...\}$

원소 a_i 와 그 원소 하나만으로 이루어진 1-tuple (a_i)는 동치로 취급.
또한 $((), a_i) = (a_i) = (a_i, ())$ 이것들도 모두 동치로 취급.

$A^0 \times A^1 = \{()\} \times \{a_1, a_2, ...\} = \{(a_1), (a_2), ...\}$

함수와 관계 functions and relations
함수(function)는 각 원소를 빠짐없이 정확히 하나씩의 원소에 일대일로 대응시키는 관계.
대응되는 대상 자체는 겹쳐도 됨. $x_1 \neq x_2$ 이더라도 $f(x_1) = f(x_2)$ 일 수 있음.

같은 x를 서로 다른 원소에 대응시키면 안 됨. $f(x) = y_1 \neq y_2 = f(x)$ 이면 f는 함수가 아님.

빠지는 게 있어도 안됨. $A = \{1, 2, 3\}$일 때, 함수 $f : A \to B$를 순서쌍의 집합으로 표현했을 때
$f = \{(1, b_1), (3, b_3)\}$ 인 경우 함수가 아님. (원소 2가 대응 안됨)
이렇게 일대일 조건을 만족하면 부분함수(partial function). 이런 부분함수에 해당하는 구조를 데이터구조 이론이나 프로그래밍언어 라이브러리에서 딕셔너리(dictionary) 또는 맵(map)이라는 이름으로 부름.

빠짐없이 대응이 정의된 함수를 전함수(total function)라고도 부름. 그냥 함수라고 하면 전함수를 뜻함. 참고로 전함수도 부분함수임. 일대일 조건을 만족하는 게 부분함수의 정의이므로, 다른 조건을 좀더 만족한다고 해서 정의상 부분함수가 아니게 되는 것은 아니다.

함수의 리턴값이 진리값(참 또는 거짓)인 함수를 불리언 함수(boolean function)라고 부르지만 predicate(술어) 또는 proerty(성질)이라고도 부름.

그냥 집합 2개 A와 B 사이의 관계(relation)는 $A \times B$의 부분집합이면 아무거나 됨.

함수와 관계의 항수(arity)
- k항 함수(k-ary function) $f : A_1 \times A_2 \times ... \times A_k \to B$에 넘겨주는 값들의 집합이 $A_1 \times A_2 \times ... \times A_k$로 k개의 곱집합. 마지막 공역(codomain) B는 항수를 따질 때 포함 안됨.
- k항 관계(k-ary relation) $R \subseteq A_1 \times A_2 \times ... \times A_k$은 마지막까지 총 k개 곱집합의 부분집합

k항 함수는 (k항 관계가 아니라) $k+1$항 관계의 특별한 경우.

헷갈리지 않도록 주의!!! k항 관계인지 검사하는 함수(즉, predicate / property)의 항수는 k다. 주어진 k-tuple이 k항 관계 $R \subseteq A_1 \times A_2 \times ... \times A_k$를 만족하나 검사(즉, 진리값을 리턴)하는 함수 $p : A_1 \times A_2 \times ... \times A_k \to \mathcal{B}$의 정의는 다음과 같다. (단, $\mathcal{B} = \{\text{TRUE}, \text{FALSE}\}$)

$$p(a_1, a_2, ..., a_k) = \begin{cases} \text{TRUE} & (a_1, a_2, ..., a_k) \in R \\ \text{FALSE} & (a_1, a_2, ..., a_k) \notin R \end{cases}$$

관계의 표기법

관계 $R \subseteq A \times B$에서 일반적으로는

- $(a, b) \in R$이면 $R(a, b)$로 표기
- $(a, b) \notin R$이면 $\neg R(a, b)$로 표기

위의 표기법은 항수에 관계없이 활용 가능

특별히 항수(arity)가 2인, 즉 이항 관계(binary relation)일 때는
중위(infix) 표기법도 활용 가능함

- $(a, b) \in R$이면 $a \, R \, b$로 표기
- $(a, b) \notin R$이면 $\neg(a \, R \, b)$ 또는 $a \not R \, b$ 등으로 표기

그래프 Graph

그래프는 두 점(vertex)들을 선(edge)으로 연결하는 구조 $G = (V, E)$

- 무방향그래프 undirected graph
  - 보통 그냥 그래프 하면 일반적인 이론(이산수학)에서는 이거
  - 선들의 집합E를 순서없는쌍(unordered pair)으로 나타낼 수도 있고
  - 무방향그래프를 방향그래프로 나타낼 수 있음: 즉, $a \longrightarrow b$와 $b \longrightarrow a$ 한쌍을 항상 같이 그리기, 한쪽만 그리지 말고. 이렇게 할 때는 E를 순서쌍의 집합으로 나타내되, E가 대칭 관계(symmetric relation)가 되도록 구성해야
- 방향그래프 directed graph
  - 그림으로 그렸을 때 선이 화살표로 표시됨
  - 방향그래프는 무방향그래프와 달리 $E \subseteq V \times V$가 대칭 관계일 필요 없음.

그래프의 선에 뭔가 라벨/꼬리표(label)를 붙인 것을 Labeled Graph 라벨된 그래프

라벨이 없는 방향그래프가 $G = (V, \{(a, b), (b, c), ...\})$ 이런 식이라면
라벨된 방향그래프는 이런 것 중 한 방식으로 표시 (edge를 triple, 즉 3-tuple에 해당하는 걸로)

- $G = (V, \{(a, l_1, b), (b, l_2, c), ...\})$ $\Longleftarrow$ 우리 교재에선 이거
- $G = (V, \{(a, b, l_1), (b, c, l_2), ...\})$
- $G = (V, \{(l_1, a, b), (l_2, b, c), ...\})$

우리 수업에서는 라벨된 방향그래프를 주로 다룸

문자열과 언어

알파벳 alphabet

- 문자열을 만들기 위한 기본 단위인 사용 가능한 글자(character) 혹은 심볼(symbol)의 집합
- 보통 Σ로 표시 (나중에는 Γ로 표시하는 것도 나오는데 그건 나중에)
- 일반적인 글자/심볼을 대표하는 변수는 $a, b, c, d \in \Sigma$ 처럼 같은 영문 알파벳 앞쪽 이탤릭체
- 구체적인 글자/심볼의 예시는 $\texttt{0}, \texttt{1} \in \Sigma_1$나 $\texttt{b}, \texttt{c} \in \Sigma_2$처럼 고정폭/타자기 폰트로

어떤 알파벳으로 이루어진 문자열 string over an alphabet

- 가능한 모든 문자열의 전체 집합 $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup ...$
- 일반적인 문자열을 대표하는 변수는 $x, y, z, w \in \Sigma^*$처럼 영문 알파벳 뒷쪽 이탤릭체

문자열은 곱집합의 원소이므로 원래는 tuple 표기법이지만 간략화된 문자열 표기법을 주로 활용. 영어 알파벳이라면 예를 들어 $abc = (a, b, c) \in \Sigma^3$ 이런 식으로 원래는 순서쌍이지만 문자열로 취급하는 경우 괄호와 컴마 생략하고 표기.

문자열의 길이(length)는 문자열에 포함된 글자/심볼의 개수로 정의하며, x의 길이를 $|x|$로 표기

길이 0개짜리 문자열(프로그래밍언어에서 보통 `""`)은 이런 이론에서는 ε으로 표기

이이붙이기(concatenation 동사로는 concat) 표기

- 두 문자열 x와 y를 이어붙인 문자열을 xy 혹은 $x \cdot y$로 표기

(형식)언어 langauge

- (같은 알파벳을 기반으로 만든) 문자열의 집합.즉, $L \subseteq \Sigma^*$

주교재 1 정규언어 Regular Languages

주교재 1.1 유한 오토마타 Finite Automata

Figure 1.1의 이해에 도움이 되는 Reddit에 올라온 참고 이미지

- https://www.reddit.com/r/nostalgia/comments/gl8xws/
- 문 열리고 지나가기 전에 (about to walk through) 밟는 패드/매트가 “front pad”
- 문 열린 쪽으로 나가서 (pass all the way through) 밟는 패드/매트가 “rear pad”

Figure 1.4는 상태전이도(state transition diagram, 줄여서 state diagram)의 예시

- state transition diagram은 방향그래프(directed graph)의 한 종류
- 라벨된 선/에지(edge)에 해당하는 화살표를 transition(상태전이)라고 부름
- 출발과 도착점이 같지만 라벨이 다른 상태전이 여러 개를 원래 각각 따로 그려야 하지만 하나로 겹쳐 모아 그리고 여러 라벨을 나열하여 표시하기도 함 (그림에서 `0`, `1`로 라벨된 화살표)
- 그래프의 점(vertex)에 해당하는 상태(state)는 원으로 표시
- 상태들 중에서 하나의 시작/초기(start/initial) 상태를 허공에서부터 가리키는 화살표로 표시 (그림에서 q_1. 언제나 딱 하나만 있어야)
- 상태들 중에서 최종/수락(final/accept state) 상태들은 테두리가 두 겹인 원으로 표시 (그림에서 q_2. 일반적으로 여러 개 있거나 없을 수도)

유한 오토마타(Finite Automata, 줄여서 FA) 혹은
유한 상태 기계(Finite State Machine, 줄여서 FSM)는 여러 가지 방식으로 활용

- Figure 1.1-1.3처럼 FA의 각 상태마다 대응되는 실제 장치의 동작상태/모드
- 확률적 요소가 가미된 마르코프 체인(Markov Chain)은 통계적 시뮬레이션에 활용
- Figure 1.4처럼 시작부터 최종 상태까지의 전이 과정에 나타난 라벨(label)을 심볼로 취급하여 이를 이어붙인 문자열을 인식/처리하는 용도 $\Longleftarrow$ 우리 주교재에서는 주로 이거에 초점

- 결정적 유한 오토마타(Deterministic Finite Automata, 줄여서 DFA)
  - 상태전이(transition)를 함수 $\delta : Q \times \Sigma \to Q$로 표현 가능
- 비결정적 유한 오토마타(Non-deterministic Finite Automata, 줄여서 NFA)도 존재
  - 상태전이(transition)를 위와 같은 형태의 함수로는 표현할 수 없는 경우까지 표현 가능한 관계 $\delta \subseteq Q \times \Sigma \times Q$로 표현

(결정적) 유한 오토마타(FA) M의 수학적인 정의(주교재 Definition 1.5)는
5-tuple(quintuple)인 $M = (Q, \Sigma, \delta, q_0, F)$로 표현

- Q는 상태(state)들의 유한 집합
- Σ는 알파벳 (심볼의 유한 집합)
- $\delta : Q \times \Sigma \to Q$는 상태전이(transition) 함수
- $q_0 \in Q$는 시작/초기(start/inital) 상태
- $F \subseteq Q$는 최종/수락(final/accept) 상태들의 집합

Figure 1.4, 1.6의 예시의 유한 오토마타 $M_1 = (\{q_1, q_2, q_3\}, \{\texttt{0}, \texttt{1}\}, \delta_1, q_1, \{q_2\})$으로 정의한다면, 상태전이 함수 $\delta_1 : Q \times \Sigma \to Q$는 다음과 같다.

δ_1	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

$$\delta_1(q_1, \texttt{0}) = q_1$$
$$\delta_1(q_1, \texttt{1}) = q_2$$
$$\delta_1(q_2, \texttt{0}) = q_3$$
$$\delta_1(q_2, \texttt{1}) = q_2$$
$$\delta_1(q_3, \texttt{0}) = q_2$$
$$\delta_1(q_3, \texttt{1}) = q_2$$

유한 오토마타/(상태)기계 "M이 문자열 w를 수락(accept)/인식(recognize)한다"는 말은, 예컨대, $w = a_1a_2a_3$가 길이 3인 문자열이라면 $\delta(\delta(\delta(q_1, a_1), a_2), a_3) \in \{q_2\} = F_1$, 여기서 F_1은 M의 수락/종료(accept/final) 상태들의 집합. 이 예시를 차례차례 풀어 쓰자면

$\delta(q_1, a_1) = q'$

$\delta(q', a_2) = q''$

$\delta(q'', a_3) = q''' \in \{q_2\} = F_1$이면 M이 $w = a_1a_2a_3$를 수락/인식한다고 판단.

수학스러운 이론에서는 프로그래밍에서 있어보이듯 따로 용어까지 있는 함수/연산자 오버로딩(overloading)이라는 개념이 숨쉬듯이 당연

그래서 교재에서는 글자 하나를 처리하는 $\delta : Q \times \Sigma \to Q$를 문자열에 나타나는 글자/심볼에 대해 연쇄적으로 호출하는 계산과정을 같은 이름의 $\delta : Q \times \Sigma^* \to Q$로 숨쉬듯 자연스럽게 오버로딩

대략 $\delta(q, a_1 a_2 ... a_{n-1} a_n) = \delta(\delta(...\delta(\delta(q, a_1), a_2), ...), a_{n-1}), a_n)$

증명에서 활용하는 수학적귀납법의 구조에 딱 떨어지게 맞도록 귀납적으로 정의하자면

$$\begin{aligned} \delta(q, \varepsilon) &= q \\ \delta(q, aw) &= \delta(\delta(q, a), w) \end{aligned}$$

위의 귀납적 정의나 교재에 나오는 설명 및 수식에서 어느 것이 심볼/글자 하나를 처리하는 δ이고, 어느 것이 여러 개의 (즉, 0개 이상의) 심볼로 이루어진 문자열을 처리하는 δ인지 구분할 수 있어야

(형식)언어(language)를 "오토마타가 수락(accept)/인식(recognize)하는 문자열(string)"이라는 조건을 만족하는 집합으로 정의하기

$$L(M) = \{w \mid M \text{ accepts } w\}$$

오토마타 M이 어떤 언어 $A \subseteq \Sigma^*$를 인식한다(즉, M accepts A)는 말은
- 언어에 속한 모든 문자열 $w \in A$은 수락하지만
- 그밖의 다른 모든 문자열 $x \notin A$ (또는 $x \in \overline{A}$)는 수락하지 않는다는 뜻.

즉, $L(M) = A$일 때 "M이 A를 인식/수락"한다고 말한다.
(문자열에 대한 "인식/수락" 개념을 언어에 대한 것으로 확장. 일종의 개념적 오버로딩?)

정규 언어(regular language)란 어떤 (결정적) 유한 오토마타(DFA) M으로 인식할 수 있는 언어.

앞으로 책에서 다음 세 방식으로 만들어지는 언어들이 정확히 똑같은 범주의 언어들이라는 것을 증명하는 내용으로 진행

1. DFA로 인식할 수 있는 언어
2. 유한집합을 모두 포함하며 다음 정규 연산들(교재 Definition 1.23: Regular Operations)에 대해 닫혀 있는 언어
   - $A \cup B$ 합집합(union)
   - AB 또는 $A \circ B$ 연결(concatenation). 이건 곱집합 $A \times B$와 마찬가지
   - A^* 몇번이고 원하는 만큼 반복해 연결(Kleene star 또는 그냥 star)
3. 정규식으로 정의할 수 있는 언어 (이건 2번을 간결한 문법으로 표현한 것일 뿐)

참고로 다른 교재들에서는 2번이나 3번을 정규 언어의 정의로 약속하고 나머지가 똑같다는 식으로 책의 내용을 구성하기도 함.

우리는 당분간 1번의 언어가 2번에 나타나는 연산들에 대해 닫혀 있음을 보이는 데 집중할 예정.
근데 이게 뭘 증명하는지 큰 그림을 파악할 필요!!!!!!
- 1번(DFA로 인식 가능한 언어들)이 2번(정규연산에 닫혀있는 언어들)에 모두 포함 ???
- 2번(정규연산에 닫혀있는 언어들)이 1번(DFA로 인식 가능한 언어들)에 모두 포함 ???

교재 Theorem 1.25 정규 언어가 합집합에 대해 닫혀 있다는 정리

증명은 대략 다음과 같은 방향으로 진행

1. A와 B가 정규언어이면 정규언어의 정의에 따라
   $A = L(M_1)$이고 $B = L(M_2)$인 DFA M_1과 M_2가 존재
2. $A \cup B = C$를 DFA 2개를 동시에 활용해서 인식하는 것은 다음과 같이 가능한데
   - $s \in C$를 M_1이 인식하는지 한번 돌려보고
   - 같은 s를 M_2가 인식하는지 또 한 번 돌려봐서
   - 둘 중 하나만 인식에 성공하면 합격!

   그런데 문제는 이런 방식은 정규 언어의 정의와 정확히 맞지는 않음.
   왜냐하면 정규언어란 정확히 1개의 DFA로 인식을 해야 되니까
3. 그러니까 방금 위에서 두 개의 기계로 하던 일을 똑같이 할 수 있는 하나의 DFA M이 존재해서
   이거 하나로 C를 인식하는, 즉 $L(M) = C$가 되는 M을 찾으면(만들어내면) 증명이 되는 것

구체적인 증명 방식, 즉 M을 만들어내는 방식은 대략 이렇게

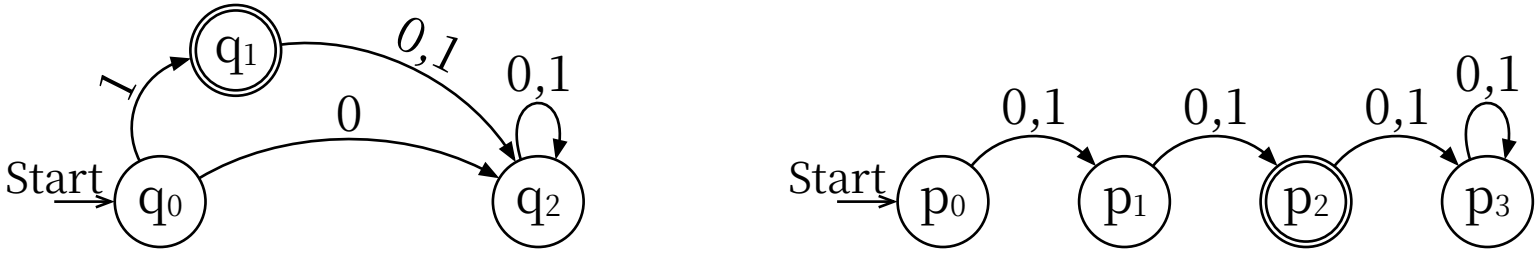
$M_1 = (Q_1 = \{q_0, q_1, ...\}, \Sigma, \delta_1, q_0, F_1)$
$M_2 = (Q_2 = \{p_0, p_1, ...\}, \Sigma, \delta_2, p_0, F_2)$ 로부터 다음을 만들어내기

$M = (Q = Q_1 \times Q_2, \Sigma, \delta, (q_0, p_0), F = \{(r_1, r_2) \mid r_1 \in F_1 \vee r_2 \in F_2\})$

여기서 $\delta((q, p), a) = (\delta_1(q, a), \delta_2(p, a))$

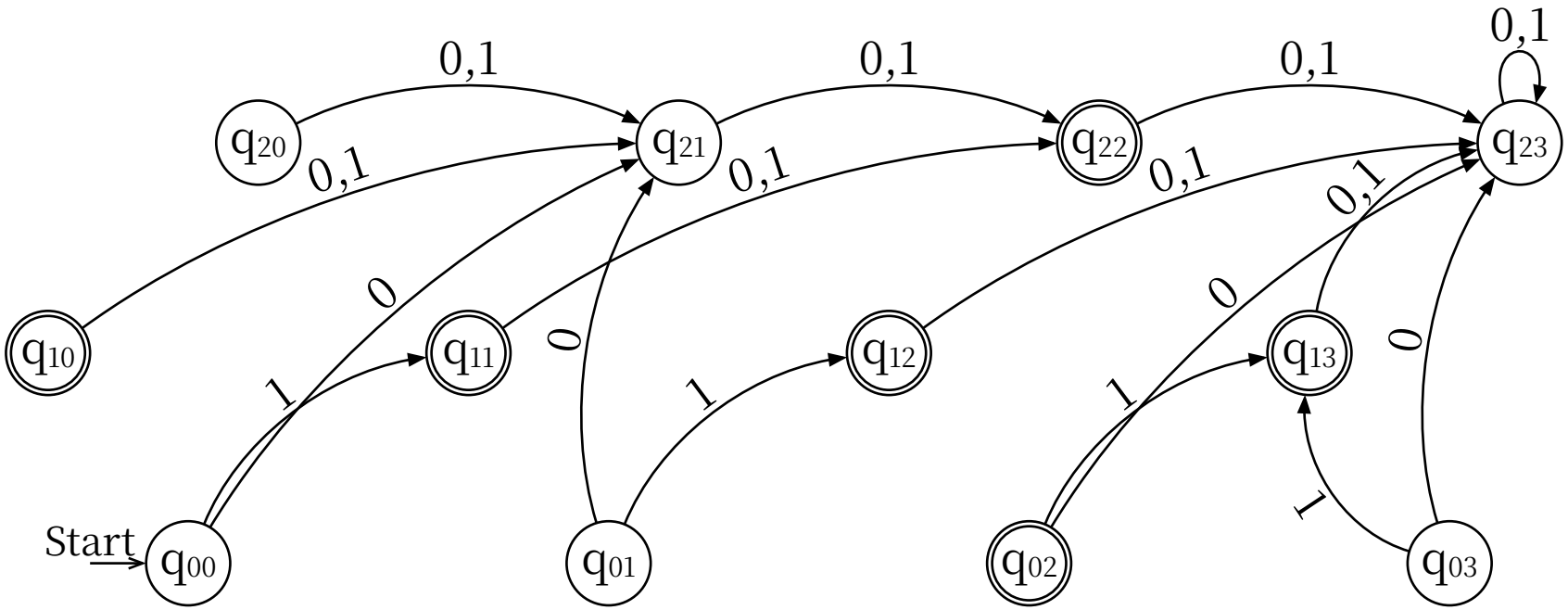
합집합 예시

알파벳 $\Sigma = \{\texttt{0}, \texttt{1}\}$의 문자열 `1`만 수락하는 $M_1 = (Q_1 = \{q_0, q_1, q_2\}, \Sigma, \delta_1, q_0, F_1 = \{q_1\})$와 같은 알파벳의 길이 2인 문자열만 모두 수락하는 $M_2 = (Q_2 = \{p_0, p_1, p_2, p_3\}, \Sigma, \delta_2, p_0, F_2 = \{p_2\})$



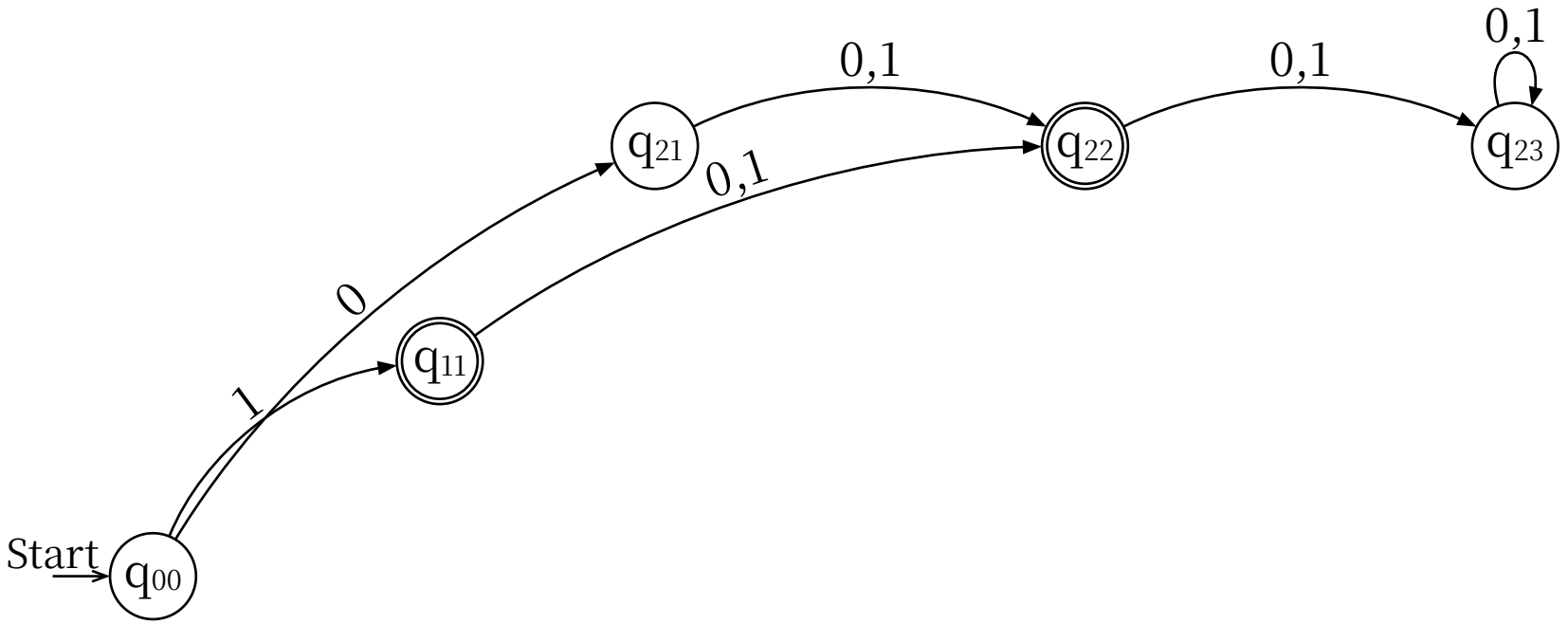
여기서 q_2나 p_3는 나머지 심볼들이 얼마만큼 더 들어오든 수락되지 못하도록 처리하는 상태인데, 이런 상태를 흡수/침몰/불능/죽은/함정/나락 상태(sink/dead/trap state)라고도 부르며 이를 강조하기 위해 q_d나 q_d처럼 아래첨자 d를 써서 표기하기도 함

$L(M) = L(M_1) \cup L(M_2)$인 $M = (Q_1 \times Q_2, \Sigma, \delta, (q_0, p_0), F = F_1 \times Q_2 \cup Q_1 \times F_2)$를 그리면



근데 여기서 필요 없는 상태들이 있으니까 지워보자

필요 없는(즉, 시작 상태에서 도달 불가능 unreachable) 상태 제거 연습



같은 방식(즉, DFA만)으로 연결(concatenation)에 대해 닫혀있음을 보이기 어려운 이유

대략 이런 방향으로 진행해야

1. A와 B가 정규언어라면 정의에 따라 $L(M_1) = A$이고 $L(M_2) = B$인 DFA M_1과 M_2가 존재
2. 어떤 문자열 w를 잘 분해해서(모든 가능한 경우 중 하나라도 되는 게 있으면 ok) $w = xy$로
   - 앞부분 x는 M_1한테 넘겨서 윙 돌려서 accept 되나 확인하고 된다면 다음으로 넘어가서
   - 뒷부분 y는 M_2한테 넘기서 윙 돌려서 accept 되는지 확인

   이렇게 되면 $AB = C$에 w가 속한다고 이런 식으로 DFA 2개를 활용해서 판단 가능
3. 문자열 w를 나눌 때 최대 개수가 한정된 유한한 최대범위 내의 경우의 수 이내로만 따질 수 있다면, 이걸 혼자 처리하는 한개짜리 DFA M을 만들 수 있을 거 같기는 한데 …

모든 가능한 경우를 고려한다는 게 최대치가 얼마인지 정하기가 어려움
- A와 B가 둘다 유한집합이면 쉽게 모든 가능한 경우의 최대 분기 개수를 한정할 수 있음
- 문제는 A와 B가 무한집합일 수도 있다는 점

즉, 2번 과정에서 w의 길이가 길어지면 길어지수록 고려해야 할 경우의 수가 그 길이에 비례해서 많아지므로 어디까지를 경우의 수 최대값 한도로 정하기 어려움.
경우의 수 최대값 한도가 정해지지 않으면 M의 상태 집합을 최대 몇개로 잡아야 할지 …
유한오토마타는 상태 집합이 유한집합이어야 하는데 바로 그게 문제

주교재 1.2 비결정성 Non-determinism

일반적인 의미로는 비결정적(non-deteriministic)이라고 하면 결정적인 경우를 제외한 나머지

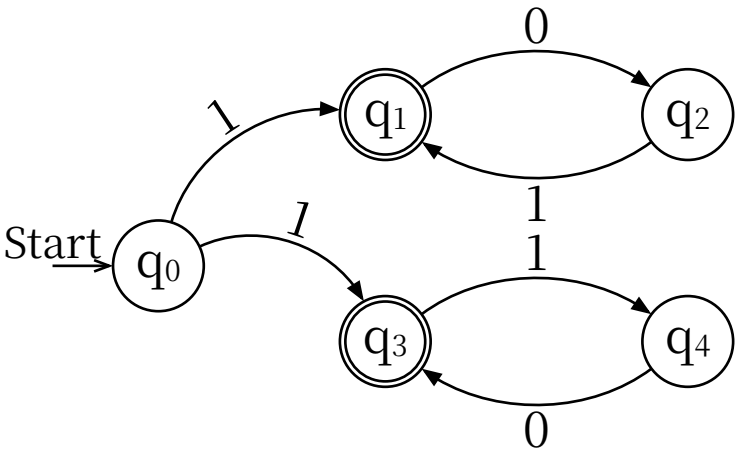
하지만 "비결정적 유한오토마타"(NFA)는 "결정적 유한오토마타"(DFA)의 개념을 사실상 포함

- NFA: $(Q, \Sigma, \delta, q_0, F)$에서 δ가 전이"관계" transition relation
- DFA: $(Q, \Sigma, \delta, q_0, F)$에서 δ가 전이"함수" transition function

함수(function)은 관계(relatioin)의 일종(함수 = 전단사관계; one-to-one 단사, onto 전사)

그러니까, 모든 DFA는 NFA 형태로 쉽게 옮겨적을 수 있음. 함수를 관계로 표현하기만 하면 됨.

NFA 예시: 첫 글자 `1`이 나온 다음에, `01`이 (0회 이상) 반복 또는 `10`이 (0회 이상) 반복



화살표에 대응되어 처리되지 않는 입력은 대략 DFA의 흡수/죽음/함정/나락 상태(sink/dead/trap state)에 해당

여러 선택 가능한 경로들 중 하나라도 수락/최종 상태(accept/final state)에 도달하면 됨

https://static0.cbrimages.com/wordpress/wp-content/uploads/2019/05/doctor-strange-avengers-infinity-war.jpg°
“어벤저스: 인피니티워” 영화 중에서, 1400만개의 미래 중 타노스를 막을 수 있는 단 1개라도 …

교재 Figure 1.28, Figure 1.29 참고

δ가 관계(relation)인 것 말고 또 다른 NFA의 추가 기능: ε전이(ε-transition)

- 그림에서는 화살표 라벨에(label)에 ε 표기
- 엡실론 생각하지 않으면 $\delta \subseteq Q \times \Sigma \times Q$ 일텐데
- 엡실론 전이를 허용하면 $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$
  (참고로 교재에서는 $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$이라는 기호로 짧게 표시)
- 원래 오토마타는 글자/심볼 하나를 보고/처리하고 화살표를 따라 수락하지만
- ε이 붙어있는 화살표는 글자/심볼 처리하지 않고 그대로 두고 화살표를 따라 이동 허용
- 활용 방식
  - 출발/도착이 같은 다른 글자/심볼과 함께 쓰면 (교재 Figure 1.27)
    그 글자가 있어도 되고 없어도 된다는
  - 수락/종료(accept/final) 상태를 항상 1개로 만들 수 있음
    원래 NFA의 수락상태들 일반상태로 만들고 거기서 새로운 하나의 수락상태로 엡실론 전이 추가
- 다른 교재나 자료들에서는 엡실론 전이를 점선 화살표로 강조하기도 함

이런 엡실론 전이를 포함한 NFA가 인식하는 언어는
- 합집합(union)에 대해 닫혀있음을 DFA의 경우보다 더 간단히 증명할 수 있음
- 연결(concatenation)에 대해 닫혀있음을 증명하기 쉬워짐
  (지난번에 DFA로는 어떻게 할지 엄두가 잘 안났었음)
- 0회 이상 반복(Kleene star 또는 그냥 star)에 대해 닫혀있음을 증명하기 쉬워짐
  (DFA로는 시도조차 안해봄)

근데 NFA는 DFA가 아니잖아? …

NFA가 할 수 있는 일이 많아 보여도 사실 DFA가 할 수 있는 범위의 일밖에 못한다 …
그러니까 쌤쌤(동치/둥가 equivalence)이다 <== 이것까지 증명하면 …..

DFA가 인식하는 언어(즉, 정규언어)가 정규 연산(즉, 합집합, 연결, 스타)에 대해 닫혀있다는 것을 증명하는 셈

NFA가 정규 연산에 대해 닫혀있음을 보이는 증명 자체는 그림만 보면 따라가며 이해하기 쉽다.
- 교재 Figure 1.46: 합집합 연산에 대해 닫힘을 증명하는 것이 DFA경우보다 이해하기 쉬움
- 교재 Figure 1.48: 연결 연산에 대해 닫힘 증명은 DFA만으로는 잘 엄두가 나지 않던
- 교재 Figure 1.50: 스타 연산에 대한 닫힘 증명에 ε전이가 특히 요긴하게 활용됨

진짜 본체는 NFA가 사실은 DFA와 동등한 능력치를 가졌을 뿐이라는 것을 증명하는 과정!!!

NFA와 DFA는 동치/등가(equivalent)

일단 NFA는 DFA보다 기능이 많기 때문에 당연히
- DFA로 할 수 있는 일(즉, 언어 인식)은 모두 NFA로 할 수 있음
- 그러니까 DFA로 인식할 수 있는 모든 언어는 NFA로도 인식할 수 있다는 이야기
- 왜냐하면 DFA의 함수 형태 δ를 관계 형태로 바꿔 표현하기만 하면 NFA가 되므로

동등하려면, 반대로 NFA로 가능한 모든 일을 DFA로도 할 수 있어야 된다는 말
- 즉, 아무 NFA 하나를 주면 그거랑 똑같은 일을 하는 DFA를 만들 수 있어야!
- 핵심 아이디어: 관계 형태 δ를 함수 형태로 바꿀 수만 있다면!!!!

이해를 쉽게 하기 위해 ε이 없는 NFA만을 일단 고려해 보자.

지금까지 함수(fucntion)를 관계(relation)로 이해/표현하곤 했는데, 이번엔 반대로
- 관계 $\delta \subseteq Q \times \Sigma \times Q$ 와 동등한 내용을
- 함수 $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ 로 변환해서 표현 (아래에서 좌변의 δ는 함수, 우변의 δ는 관계)

$$\delta(r, a) = \{\, r' \mid (r, a, r') \in \delta \,\}$$

r에서 라벨(label) a를 따라 도착 가능한 모든 상태 r'의 "집합"을 함수 $\delta(r, x)$의 결과값으로 정의
(참고로, 교재 Definition 1.37의 NFA 정의가 애초에 이런 식으로 되어 있음)

멱집합 $\mathcal{P}(Q) = \{X \mid X \subseteq Q\} = 2^Q$ 모든 부분집합의 집합. 다른 교재나 자료에서는 2^Q로도 표기.

ε전이가 없는 NFA의 관계 $\delta \subseteq Q \times \Sigma \times Q$는 함수 $\delta : Q \times \Sigma \to \mathcal{P}(Q)$로 표현 가능
하지만 이건 우리가 원하는 정확한 DFA의 전이함수 형태가 아님

DFA의 전이함수 형태는 $\delta' : Q' \times \Sigma \to Q'$ 첫째 파라메터와 리턴 타입이 같아야
즉, $Q' = \mathcal{P}(Q)$가 되도록 맞춰줘야 $\cdots$ $\delta' : \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$

$$\delta'(R, a) = \{\, r' \mid r \in R, r' \in \delta(r, a) \,\}$$

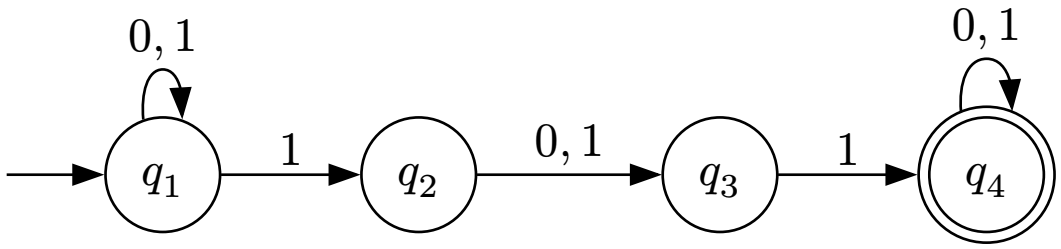
NFA의 시작 상태가 q_1라면 DFA의 시작 상태는 $\{q_1\}$

NFA의 수락상태집합이 F라면 DFA의 종료상태집합 $F' = \{R \mid R \in Q', F \cap R \neq \varnothing\}$

DFA의 상태 하나에 해당하는 멱집합의 원소 $R \in \mathcal{P}(Q)$ 즉 $R \subseteq Q$로 가능한 모든 경우의 수는 $2^{|Q|}$

원래 NFA의 상태 Q가 k개라면, 똑같은 일을 할 수 있는 DFA를 상태 2^k개 이하로 항상 구성 가능!!!

- 단순무식한 방법:
  교재 증명 과정 곧이곧대로 모든 부분집합을 DFA 상태로, 즉 Q'를 Q의 멱집합으로 설정
  - Prolog로 멱집합 계산하는 코드는 §1.7 을 참고
  - 문제는, DFA 시작 상태로부터 도달 불가능한 불필요한 상태들이 많이 포함될 수 있음
    (그래프를 알아보기 좋게 혹은 더 효율적으로 구성하려면 불필요한 상태들을 정리해야)
- 조금 더 효율적인 방법:
  애초에 DFA 시작 상태 $\{q_1\}$로부터 도달 가능한 상태만 NFA 전이관계로부터 계산



- $\underline{\{q_1\}}$
- $\{q_1\}, \underline{\{q_1, q_2\}}$ $\quad\quad \{q_1\} \xrightarrow{0} \{q_1\}, \quad \{q_1\} \xrightarrow{1} \underline{\{q_1, q_2\}}$
- $\{q_1\}, \{q_1, q_2\}, \underline{\{q_1, q_3\}}, \underline{\{q_1, q_2, q_3\}}$ $\quad\quad \cdots, \{q_1, q_2\} \xrightarrow{0} \underline{\{q_1, q_3\}}, \ \{q_1, q_2\} \xrightarrow{1} \underline{\{q_1, q_2, q_3\}} \cdots$
- $\{q_1\}, \{q_1, q_2\}, \{q_1, q_3\}, \{q_1, q_2, q_3\}, \underline{\{q_1, q_2, q_4\}}, \underline{\{q_1, q_2, q_3, q_4\}}$

  $\cdots \{q_1, q_3\} \xrightarrow{1} \underline{\{q_1, q_2, q_4\}},$

  $\{q_1, q_2, q_3\} \xrightarrow{1} \underline{\{q_1, q_2, q_3, q_4\}}, \cdots$
- $\cdots \cdots \cdots \underline{\{q_1, q_3, q_4\}}$ 아마도 여기서 이거 하나 말고는 더 안나올 듯?

계속 이렇게 진행하다 보면 언젠가는 더 이상 새로운 DFA 상태(밑줄 친 부분)가 나오지 않음

단순무식하게 모든 멱집합을 다 고려해 NFA를 DFA를 변환하는 방식의 Prolog 코드부터 살펴보자.

nfa2dfaNOeps1.pl

```
subset(    [],    []).
subset([X|Xs],[X|Ys]) :- subset(Xs,Ys).
subset(   Xs ,[_|Ys]) :- subset(Xs,Ys).

% NFA 정의
state(q1). state(q2). state(q3). state(q4).        % 상태
symbol(0). symbol(1).                             % 알파벳
delta(q1,0,q1).  delta(q1,1,q1). delta(q1,1,q2).  % 전이관계
delta(q2,0,q3).  delta(q2,1,q3).
delta(q3,1,q4).                   delta(q4,0,q4).  delta(q4,1,q4).
start(q1).                                        % 시작상태
final(q4).                                        % 수락상태

?- findall(X,state(X),Q), format('all states of NFA: Q = ~w~n', [Q]).
?- findall(X,state(X),Q), findall(R,subset(R,Q),PS),
   format('all subsets (powerset) of Q: ~w~n', [PS]).
```

```
O: all states of NFA: Q = [q1,q2,q3,q4]
O: all subsets (powerset) of Q: [[q1,q2,q3,q4],[q1,q2,q3],[q1,q2,q4],[q1,q2],[q1,q3,q4],
[q1,q3],[q1,q4],[q1],[q2,q3,q4],[q2,q3],[q2,q4],[q2],[q3,q4],[q3],[q4],[]]
```

NFA 정의를 바탕으로 Prolog 규칙(rule)을 통해 DFA의 정의들이 계산되도록 하자

nfa2dfaNOeps1.pl

```
% DFA 정의
stateD(R) :- findall(X,state(X),Q), subset(R,Q).  % DFA 상태로 Q의 부분집합을 모두 인정
startD([X0]) :- start(X0).                        % DFA 시작상태
finalD(R) :- stateD(R), final(X), member(X,R).    % DFA 수락상태
deltaD(R,A,R1) :- stateD(R), stepD(R,A,R1), stateD(R1). % 전이함수
% stepD는 NFA 상태집합 Q의 부분집합인지 검사 없이 집합R에 대해 NFA 전이관계 delta 적용
stepD(R,A,R1) :- findall(Y, ( member(X,R), delta(X,A,Y) ), R1).

?- stepD([q1,q10], 1, R1) -> format('stepD([q1,q10],...) success: ~w~n', [R1])
                           ; format('stepD([q1,q10],...) failure ~n').
?- deltaD([q1,q10], 1, R1) -> format('deltaD([q1,q10],...) success: ~w~n', [R1])
                            ; format('deltaD([q1,q10],...) failure ~n').

?- findall(R, finalD(R), Rs), format('DFA 수락상태집합: ~w~n', [Rs]).
```

```
O: stepD([q1,q10],...) success: [q1,q2]
O: deltaD([q1,q10],...) failure
O: DFA 수락상태집합: [[q1,q2,q3,q4],[q1,q2,q4],[q1,q3,q4],[q1,q4],[q2,q3,q4],[q2,q4],
[q3,q4],[q4]]
```

수동으로 Prolog 쿼리를 여러 개 시도하며 돌려보는 효율적인 DFA 상태 생성 아이디어

nfa2dfaNOeps1.pl

```
?- startD(R0), stepD(R0,1,R1), write(R1), nl.
?- startD(R0), stepD(R0,1,R1), stepD(R1,0,R2), write(R2), nl.
?- startD(R0), stepD(R0,1,R1), stepD(R1,0,R2), stepD(R2,1,R3), write(R3), nl.
?- startD(R0), stepD(R0,1,R1), stepD(R1,1,R2), write(R2), nl.
?- startD(R0), stepD(R0,1,R1), stepD(R1,1,R2), stepD(R2,1,R3), write(R3), nl.
?- startD(R0), stepD(R0,1,R1), stepD(R1,1,R2), stepD(R2,1,R3), stepD(R3,0,R4),
   write(R4), nl.
```

```
0: [q1,q2]
0: [q1,q3]
0: [q1,q2,q4]
0: [q1,q2,q3]
0: [q1,q2,q3,q4]
0: [q1,q3,q4]
```

그런데, 개수가 많더라도 멱집합으로 상태 전체의 집합을 미리 만들어 놓고 시작하는 경우와 달리, NFA 화살표를 따라가며 계산하다 보면 중복이나 순서만 다르고 집합으로 해석하면 동일한 리스트도 나올 수 있으므로, sort를 통해 딱 하나의 대표 집합 표현으로 통일하는 것이 좋다.

nfa2dfaNOeps1.pl

```
?- startD(R0),
   stepD(R0,1,R1), stepD(R1,1,R2), stepD(R2,1,R3), stepD(R3,0,R4), stepD(R4,1,R5),
   write(R5), nl. % 중복이 발생할 수도 있네 ?!?!?!?!?!

?- startD(R0),
   stepD(R0,1,R1), stepD(R1,1,R2), stepD(R2,1,R3), stepD(R3,0,R4), stepD(R4,1,R5),
   sort(R5,S),  % sort하면 편안!!!
   write(S), nl.
```

```
0: [q1,q2,q4,q4]
0: [q1,q2,q4]
0:
```

이번에는 조금 더 효율적으로 NFA를 DFA로 변환하는 방식의 Prolog 코드를 살펴보자.

nfa2dfaNOeps2.pl

```
subset(    [],    []).
subset([X|Xs],[X|Ys]) :- subset(Xs,Ys).
subset(   Xs ,[_|Ys]) :- subset(Xs,Ys).
% NFA 정의는 이전과 같다
state(q1). state(q2). state(q3). state(q4).        % 상태
symbol(0). symbol(1).                             % 알파벳
delta(q1,0,q1).  delta(q1,1,q1). delta(q1,1,q2).  % 전이관계
delta(q2,0,q3).  delta(q2,1,q3).
delta(q3,1,q4).                   delta(q4,0,q4).  delta(q4,1,q4).
start(q1).                                        % 시작상태
final(q4).                                        % 수락상태

% stepD는 NFA 상태집합 Q의 부분집합인지 검사 없이 집합R에 대해 NFA 전이관계 delta 적용
stepD(R,A,R2) :- findall(Y, ( member(X,R), delta(X,A,Y) ), R1), sort(R1,R2).
% 여기 왜 sort를 해야 되는지 생각해 보기!!!
```

이번 DFA의 정의는 `stateD`를 `stepD` 계산을 통해 생성하는 점이 다르고 나머지 부분은 지난번 것과 같다. (다음 슬라이드에서 계속 …)

인공지능 에이전트에게 Prolog의 테이블링(talbing)에 대해 설명해 달라고 요청해 보기.

처음 보면 마법의 가루 같아 보이지만, Python에서 `functools` 패키지의 `cache`와 비슷한 느낌의 물건으로, 소위 memoization, caching, dynamic programming 등등과 관련 있는 개념인데, … 여기서는 알고리즘의 시간복잡도를 줄이는 데 응용한 것이 아니라 애초에 무한루프 돌 녀석을 못돌게 막는 데 응용함 (마치 미로에서 이미 탐색했던 길입구에 표시하고 다시 안들어가는 것처럼)

nfa2dfaNOeps2.pl

```
% 효율적인 DFA 정의 (SWI-Prolog 구현의 tabling 활용)
:- table stateD/1.  % 원래 무한루프로 빠지는 코드인데, 그걸 막는 마법의 가루같은 ...
stateD(R0) :- startD(R0).
stateD(R1) :- stateD(R), symbol(A), stepD(R,A,R1).

startD([X0]) :- start(X0).                          % DFA 시작상태
finalD(R) :- stateD(R), final(X), member(X,R).      % DFA 수락상태
deltaD(R,A,R1) :- stateD(R), stepD(R,A,R1), stateD(R1). % 전이함수

?- findall(R, stateD(R), Rs), format('DFA 상태집합: ~w~n',[Rs]).
?- findall(R, finalD(R), Rs), format('DFA 수락상태집합: ~w~n',[Rs]).
```

```
O: DFA 상태집합: [[q1],[q1,q2],[q1,q2,q3],[q1,q2,q3,q4],[q1,q2,q4],[q1,q3],[q1,q3,q4],
[q1,q4]]
O: DFA 수락상태집합: [[q1,q2,q3,q4],[q1,q2,q4],[q1,q3,q4],[q1,q4]]
O:
```

교재 Theorem 1.39의 증명에서 E로 표기하는 ε-closure
즉, 주어진 상태(들)로부터 ε전이만으로 도달할 수 있는 상태들의 집합이라는 개념을 활용하면, ε전이를 포함한 NFA로 할 수 있는 일도 모두 DFA로 할 수 있다는 증명으로 확장 가능

- DFA의 시작 상태는 $E(\{q_1\})$
  즉, NFA 시작상태 q_1로부터 NFA의 ε라벨 에지들만 0회 이상 따라서 도달 가능한 모든 곳들
- DFA의 수락상태집합 $F' = \{ R \mid R \in Q', E(X) \cap R \neq \varnothing \}$

그 다음 주교재에서는 Closed Under the Regular Operations

앞서 이야기했든 NFA를 이해했으면 그림만 그려놔도 증명이 이해되는 내용

그런데, 그 중에서 스타 연산에 대한 그림에서 ε 전이로 새로운 시작 상태를 만드는지 부분은 생각해 보고 이해할 필요가 있음.

이걸 몰라도 제시된 증명의 논리 흐름 자체는 문제가 없으므로 증명이 맞다는 걸 확인하는 건 문제가 없는데, 새로운 상태를 만들지 않으면 이 증명이 진행되지 않는다는 걸 이해해야 스타 연산에 대해 닫혀있다는 증명을 좀더 제대로 이해하는 것.

스타 연산 적용한 NFA 구성할 때 새로운 시작상태를 만들지 않으면 문제 생기는 경우

- 수락상태가 아닌 다른 곳에서 시작상태로 돌아오는 ε전이가 있을 때

주교재 1.3 정규식 Regular Expressions

정규식은 의도적으로 기존에 사용하던 기호를 많이 overloading하고 있음에 유의!

정규식의 의미는 언어(문자열 집합)다. (아래에서 R, R_1, R_2는 정규식)

1. 알파벳 심볼을 오버로딩한 정규식 a의 의미는 $\{a\}$
2. 엡실론 기호를 오버로딩한 정규식 ε의 의미는 $\{\varepsilon\}$
3. 공집합 기호를 오버로딩한 정규식 $\varnothing$의 의미는 $\{\}$
4. 합집합 기호를 오버로딩한 $R_1 \cup R_2$의 의미는,
   R_1의 의미가 언어 A_1이고 R_2의 의미는 언어 A_2일 때, 두 언어의 합집합 $A_1 \cup A_2$
   - 다른 교재나 자료에서는 $R_1 | R_2$ 또는 $R_1 + R_2$와 같이 표기하기도 함.
5. 연결(concatenation)을 오버로딩한 $R_1 \circ R_2$혹은 $R_1 R_2$의 의미는,
   R_1의 의미가 언어 A_1이고 R_2의 의미는 언어 A_2일 때, 두 언어의 연결 $A_1 \circ A_2$
   즉, $\{xy \mid x \in A_1, y \in A_2\}$
   - 다른 교재나 자료에서는 $R_1 \cdot R_2$와 같이 표기하기도 함.
6. 스타 연산을 오버로딩한 R^*의 의미는, R의 의미가 언어 A일 때, A의 스타 연산 A^*
   즉, $\{x_1 x_2 ... x_n \mid x_i \in A, n \in \mathbb{N}\}$ 또는 $A^0 \cup A^1 \cup A^2 \cup \cdots$

정규식이 영어로 regular expression이므로 약어로 regex라고 많이들 부른다.
또 헷갈리지 않는 범위에서는 RE라고 줄여서 부르는 경우도 있음.

교재 Lemma 1.55는 regex로 표현 가능하면 “DFA로 인식할 수 있다”(=정규언어의 정의)는 내용.

이건 이미 NFA와 DFA가 동등하고 NFA가 인식하는 언어들 정규 연산에 대해 닫혀 있다고 살펴본 내용으로 이미 거의 증명된 거나 마찬가지.

다시 한번 강조하지만 이 증명은 유한집합과 정규연산으로 만들어낼 수 있는 집합(언어)들이 정규언어임을 보장한다는 말. 즉 정규연산으로 만들어낼 수 있는 언어들이 모든 정규언어의 부분집합에 해당한다는 이야기.

- 교재 Lemma 1.6는 반대 방향, 그러니까 DFA로 인식할 수 있는 언어는 빠짐없이 regex로 표현 가능하다는 내용.

방법은 NFA보다도 더 기능이 강력한 GNFA(Generalized NFA)를 이용하는 것.
- GNFA는 상태 전이 라벨이 심볼 또는 ε이 아니라 정규식(regex)
- DFA를 NFA로 쉽게 변환할 수 있는 것처럼, NFA를 GNFA로 쉽게 변환할 수 있음.
  - 왜내하면 심볼과 ε에 해당하는 오버로딩된 정규식이 있기 때문
  - 똑같이 생겼는데 상태 전이 라벨이 심볼 또는 ε이 아니라 정규식(regex)으로 바뀌면 됨.

DFA $\longrightarrow$ GNFA $\longrightarrow$ RegEx 변환 과정

1. 우선 시작 상태와 수락 상태를 새롭게 만들기
   - 새로운 시작 상태를 만들기
     - 기존 시작 상태로 ε전이 화살표를 연결
     - 기존 시작 상태를 더 이상 시작 상태가 아닌 것으로 처리
   - 수락 상태를 하나로 통함
     - 새로운 수락 상태를 만들고
     - 기존 수락 상태들에서 ε전이 화살표로 연결
     - 기존 수락 상태들은 더 이상 수락 상태가 아닌 것으로 처리

2. 다음과 같이 상태, 화살표를 줄이는 과정을 반복
   - $q_1 \xrightarrow{R_1} q_2 \xrightarrow{R_2} q_3$를 $\quad q_1 \xrightarrow{R_1 R_2} q_3$ 로 줄임. (상태도 줄어들고 화살표도 줄어들고)
   - $q_1 \xrightarrow{R_1} q_2$와 $q_1 \xrightarrow{R_2} q_2$를 $\quad q_1 \xrightarrow{R_1 \cup R_2} q_2$ 로 줄임. (화살표만 줄어듦)
   - $q_1 \xrightarrow{R_1} q_2 \circlearrowleft_{R_2} \xrightarrow{R_3} q_3$를 $\quad q_1 \xrightarrow{R_1 R_2^* R_3} q_3$ 로 줄임. (상태도 줄어들고 화살표도 줄어들고)

3. 위 과정을 시작, 수락 상태 두 개만 남고 화살표는 하나만 남을 때까지 반복
   - 화살표가 하나만 남으면 그 라벨이 바로 원래 DFA를 나타내는 정규식(regex)

주교재 1.3 정규언어가 아닌 언어

수업시간에 텍스트 노트패드에다 설명함

다음에 기회가 되면 정리 내용을 추가할 수도 …

주교재 2 Context-Free Languages 문맥자유언어

정규언어(regular language)를
- 표현하기 좋은 **정규식**(regular expression, regex)
- 인식하기 위한 계산 과정을 따라가기 좋은 **유한 오토마타**(Finite Automata)

문맥자유언어(context-free language, CLF)를
- 표현하기 좋은 **문맥자유문법**(context-free grammar, CFG)
  - 사실 CFL이 바로 이런 문법(CFG)으로 생성할 수 있는 언어로 정의됨!!
- 인식/생성하기 위한 계산 과정을 따라가기 좋은 **Pushdown Automata** (PDA)

문맥자유문법(CFG)에 나타나는 심볼(symbol, 글자)의 종류
- 단말(terminal) $a, b, ... \in \Sigma$
  - 지금까지 FA에서도 다루던 입력 심볼
- 변수(variable), 비단말(non-terminal) $A, B, C, ... \in V$ (다른 교재는 V대신 N을 쓰기도)
  - 문법 규칙에만 등장. 즉, 생성 과정에서만 사용되고 최종 문자열에는 나타나지 않음

왼쪽이 같은 CFG 규칙끼리 묶어서 오른쪽에 여러 개의 선택지를 |로 구분하여 나열하는 방식의 마치 하나의 규칙처럼 압축해 표현하는 축약 표기법. 예를 들면 우리 교재 2장 첫번째 CFG 문법 규칙 예제의 두개의 문법 규칙을 이러한 축약 표기법으로 $S \longrightarrow 1S0 \mid \#$

CFG는 $G = (V, \Sigma, R, S)$ 이렇게 4-tuple로 정의됨.
- V: 변수들의 집합
- Σ: 단말들의 집합
- R: 문법 규칙(production 또는 substitution rule 이라고도함)들의 집합
- S: 시작 변수

$u \Rightarrow v$는 문자열 u에 어떤 한 문법 규칙에 따라 하나의 변수를 치환(substitution)함으로써 문자열 v를 유도할 수 있음을 나타냄. (여기서 $u, v \in V \cup \Sigma$)
이렇게 화살표 하나가 나타내는 것을 유도(derivation) 과정의 한 단계(step)라고도 말함.

$u \underset{G}{\Rightarrow} v$와 같이 G라는 CFG의 유도 단계(derivation step)임을 강조하기 위해
화살표 아래에 CFG의 이름을 적어 표기하기도 함.

CFG인 G로 정의되는 CFL을 $L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$ 이렇게 표현할 수 있다.
(즉, 0회 이상의 유도 단계를 거쳐 시작 변수로부터 유도되는 단말 문자열의 집합)

CNF(Chomsky Normal Form)

- 모든 문법 규칙이 다음 두 가지 형태 중 하나임
  0. $S \to \varepsilon$ 은 예외적으로 허용되며, 이런 경우 S가 다른 규칙의 오른쪽에 등장할 수 없음
  1. $A \to BC$ (A, B, C는 변수)
  2. $A \to a$ (A는 변수, a는 단말)

GNF(Greibach Normal Form) 우리 교재 말고 다른 교재에 등장

- 모든 문법 규칙이 다음 형태를 가짐
  0. $S \to \varepsilon$ 은 예외적으로 허용되며, 이런 경우 S가 다른 규칙의 오른쪽에 등장할 수 없음
  1. $A \to a\alpha$ (A는 변수, a는 단말, α는 변수들의 문자열)

모든 CFG를 CNF로 변환할 수 있고, 또 GNF로도 변환할 수 있음이 알려져 있음.
빈 문자열을 포함하지 않는 언어의 경우에는 위의 0번이 필요치 않음.

수업시간에 GNF 형태의 문법 규칙 하나를 여러 개의 CNF 형태의 문법 규칙들로 변환하는 예시를 다룸. 수업시간의 기억을 더듬어 $A \to aB_1B_2B_3$를 여러 개의 CNF 규칙들로 변환해 보라.

푸시다운 오토마타(PDA)

- 간단히 말하면 PDA = NFA + 스택(stack)
- $(Q, \Sigma, \Gamma, \delta, q_0, F)$로 정의됨.
  - Q, Σ, q_0, F는 NFA와 마찬가지
  - Γ: 스택 알파벳
  - δ: 전이 관계 $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \times (Q \times \Gamma_\varepsilon)$
    FA와 달리
    - 입력 심볼뿐만 아니라 스택의 최상단 심볼을 읽고
    - 다음에 어느 상태로 갈지 뿐만 아니라 스택의 최상단 심볼을 어떻게 바꿀지도 결정

  다른 교재에서는 전이관계를 $\delta : Q \times \Sigma_\varepsilon \times \Gamma \times (Q \times \Gamma^*)$ 이렇게 정의 하기도
- 우리 교재 정의는 한번의 전이(화살표)로 인한 스택의 크기 변화 최대 1씩만
  - 정규화된 CFG의 규칙을 대응시킬 수 있음
- 다른 교재의 그런 정의는 한번의 전이로 스택의 크기 변화가 한번에 크게 일어나는 것 허용
  - 정규화되지 않은 CFG의 규칙을 한번에 하나의 전이로 처리하기 편함

수업시간에 CNF 문법규칙을 어떻게 PDA에서 실행시키는지 다루긴 했으나, 사실 한 두 단계 거쳐 가는 중간과정이 있는데, 지난 수업시간에는 정말 대략적인 느낌만 다룸.

여기서는 좀더 구체적으로 단계를 밟가가며 CNF 문법 규칙을 PDA에서 실행시켜 보자.

$A \longrightarrow a$ 규칙 처리 과정 (수업시간엔 이걸 화살표 하나처럼 대충 설명했음)

q_1 `입력: __a???  스택:[A|...]`

$\downarrow \varepsilon, A \to a$ expand (스택 가장 위에서, 문법규칙 왼쪽 A를 오른쪽 a로 치환)

q_2 `입력: __a???  스택:[a|...]`

$\downarrow a, a \to \varepsilon$ match (스택의 최상단 심볼 a와 입력 심볼 a가 일치하여 소비됨)

q_3 `입력: ___???  스택:  [...]`

$D \longrightarrow BC$ 규칙 처리 과정 (이건 수업시간에 다룬 그대로 expand가 여러 단계에 걸쳐 일어남)

q_4 `입력: __????  스택:  [D|...]`

$\downarrow \varepsilon, D \to C$

q_5 `입력: __a???  스택:  [C|...]`

$\downarrow a, \varepsilon \to B$

q_6 `입력: ___???  스택:[B|C|...]`

방금 이전 장의 설명이 소위 말하는 CFG를 Top-Down 방식으로 PDA에서 실행시키는 과정

그런데 지금까지 설명에서는 CFG를 보고 PDA를 구성할 때 NFA에 해당하는 부분(즉, 교재 그림 2.22의 control)의 상태가 몇 개 필요한지, 어떻게 구성하는지가 명확히 설명되지는 않음.

그 부분을 설명하는 것이 바로 교재 그림 2.23, 2.24, 2.26

그림 2.23, 2.24를 보면 우리 교재도 사실상 다른 교재 PDA의 $\delta : Q \times \Sigma_\varepsilon \times \Gamma \times (Q \times \Gamma^*)$ 형식의 전이 관계에 해당하는 개념을 설명을 위한 도구로 사용하는 것이라 볼 수 있다.

그림 2.23: CFG의 문법규칙 하나를 PDA에서 실행시키는 과정을 그림으로 설명
- 왼쪽: CFG의 문법규칙 하나를 PDA의 q_{loop}상태에서 출발해서 다시 돌아오는 경로 하나로 요약 (다른 교재의 한꺼번에 스택에 여러 개를 쌓는 전이 단계를 허용하는 버전의 PDA라면 실제로 이 화살표 하나가 구체적인 전이 단계 하나에 해당)
- 오른쪽: 우리 교재의 PDA 정의에 충실하게 중간 단계 상태들을 거치며 한번에 하나의 변수만 스택에 쌓으며 q_{loop}으로 돌아오는 여러 단계의 전이 과정을 자세히 표현

그림 2.24는 가장 중요한 3개의 상태들을 중심으로 CFG를 top-down 방식으로 실행시키는 PDA의 전체적인 구조를 개략적으로 요약하여 보여주는 그림

그림 2.26: CFG를 top-down 방식으로 실행시키는 PDA의 전체 구조를 CFG규칙 expand과정의 중간 단계까지 하나하나 나열하며 match 과정 전이까지 모두 완전하게 보여주는 예시 그림

그림 2.24와 2.26에서 $\$ \in \Gamma$는 스택의 밑바닥 끝임을 나타내는 심볼(기호, 글자)

pda_naive.pl

```
% (1) S -> 0S1  (2) S -> x  두 CFG 규칙의 top-down 방식 PDA
state(q_start). state(q_s2l). state(q_loop). state(q_accept).
state(q_1__1). state(q_1_S1).                          % Q (states)
sigma(0). sigma(1). sigma(x).                          % Σ
gamma('$').    gamma('S').    gamma(X) :- sigma(X).    % Γ
% 수식으로 된 수학적 정의보다는 PDA 전이도(그림)의 구조와 맞게 delta 작성
delta(q_start, [],    []-['$'], q_s2l).
delta(q_s2l,   [],    []-['S'], q_loop).
delta(q_loop,  [], ['S']-[ 1 ], q_1__1).  % rule (1) __1
delta(q_loop,  [],    []-['S'], q_1_S1).  % rule (1) _S1
delta(q_loop,  [],    []-[ 0 ], q_loop).  % rule (1) 0S1, back to q_loop
delta(q_loop,  [], ['S']-[ x ], q_loop).  % rule (2)
delta(q_loop, [A], [ A ]-[],    q_loop) :- sigma(A).  % match terminal
delta(q_loop,  [], ['$']-[],    q_accept).

?- delta(q_start, [], S1-S2, Q2), delta(Q2, [], S3-S4, Q3),
   format("~w ~w ~w ~w~n", [S1-S2, Q2, S3-S4, Q3]).
?- delta(q_loop, [], ['S']-[1], Q2), delta(Q2, [], S3-S4, Q3),
   format("~w ~w ~w ~w~n", [['S']-[1], Q2, S3-S4, Q3]).
```

```
O: []-[$] q_s2l []-[S] q_loop
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/pda_naive.pl:18:
```

pda_naive.pl

```
% delta(Q1, A, S1-S2, Q2) 형식 검사 위한 helper predicates
sigmaE([A]) :- sigma(A).  sigmaE([]).
gammaE([X]) :- gamma(X).  gammaE([]).
% every Q1 and Q2 is a state
?- findall(Q1, delta(Q1,_,_,_), QS), maplist(state, QS).
?- findall(Q2, delta(_,_,_,Q2), QS), maplist(state, QS).
% every X are either a singleton alphabet symbol list or the empty list
?- findall(A,  delta(_,A,_,_), AS), maplist(sigmaE, AS).
% every S1 and S2 is either a singleton stack symbol list or the empty list
?- findall(S1, delta(_,_,S1-_,_), SS), maplist(gammaE, SS).
?- findall(S2, delta(_,_,_-S2,_), SS), maplist(gammaE, SS).

% delta를 0회 이상 여러 번에 걸쳐 계속 진행시키는 과정 (Q1,W1,S1)->...->(Q2,W2,S2)
steps(Q1, W1, S1, Q2, W2, S2) :- true.  % TODO 여기 true 대신 내용 작성.
                                        % 여러 규칙으로 나눠 작성해도 됨.
runPDA(Input) :- steps(q_start, Input, ['$'], q_accept, [], _).
```

```
E: Warning:    Goal (directive) failed: user:(delta(q_loop,[],
['S']-[1],_6790),delta(_6790,[],_6874-_6876,_6926),format("~w ~w ~w ~w~n",[['S']-
[1],_6790,_6874-_6876,_6926]))
E: Warning: /home/kyagrd/github/hnu-pl/TC2026/_build/pda_naive.pl:34:
O:
E: Warning:    Singleton variables: [Q1,W1,S1,Q2,W2,S2]
```

교재 2장의 CFL, CFG, PDA 관련해 이후로 나오는 내용들을 따라 공부하려 할 때 막히는 부분.

(비결정적) PDA를 먼저 배움 (이론적으로 간단하고 수학적 성질도 증명하기 좋으므로)
- CFG를 top-down 방식으로 실행하는 PDA <== 지금까지 교재를 따라가며 열심히 다룸
- CFG를 bottom-up 방식으로 실행하는 PDA ??? 이게 보통 교재들에 직접 등장을 안하는 경향 (Linz 책 최신판에는 부록 비슷하게 뒤에 추가된 장에 등장함)

나중에 파서(parser) 알고리즘 관련해 결정적 버전의 PDA인 DPDA에 대해 다루면서
- 대략 top-down 방식 DPDA 동작에 대응되는 LL 파서 <== 우리 교재에서 안다룸
- 대략 bottom-up 방식 DPDA 동작에 해당하는 LR 파서 <== ??? 우리 교재에서 다룸 ???

이런 개념들을 접하는데 …

하향(top-down)식 (비결정적) PDA 동작 원리를 알고 있으면 LL 파서 이해에 도움이 됨. 근데 이건 우리 교재에서는 안다룸. 문법분석(parsing) 자체를 주제로 한 책이나 파서의 대표적 활용 분야인 컴파일러를 주제로 하는 교재에서 다룸.

상향(bottom-up)식 (비결정적) PDA 동작 원리를 알고 있으면 LR 파서 이해에 도움이 됨.
(비결정적 PDA가 더 간단한 근본 형태이고 결정적 PDA는 그 위에 제약을 추가한 셈이기 때문)
근데 우리 교재 포함 여러 다른 교재에도 이 부분이 비어 있어서, 상향식 PDA를 상상하고 그게 rightmost deriviation과 관련이 있음을 스스로 책을 보고 깨우쳐치면 이해가 확 트이는 느낌이고 안그러면 뭔가 너무 갑자기 많은 디테일을 다루는 느낌을 받아 혼란스러울 수 있음.

그래서 여기서 우래 교재에 직접적으로 소개되지 않은 bottom-up 방식의 (비결정적) PDA 동작 원리를 알아보는 간단한 예시를 정리하면 도움이 될 것이라 생각됨.

(1) $S \to 0S1$ (2) $S \to x$ 문법규칙들로 구성된 CFG에 대한 두 방식의 비결정적 PDA 실행 예시를 비교/대조. 상태(state)는 생략하고 입력과 스택을 중심으로만 …

- Top-down 방식 (스택을 왼쪽으로 눕혀 왼쪽이 top인 형태. `||`는 바닥을 표시)

```
0x1     [S,$]||   ⊢ ⋯ ⊢     expand (1)  자잘한 중간 상태 2개를 거쳐서
0x1 [0,S,1,$]||   ⊢         match 0
_x1   [S,1,$]||   ⊢         expand (2)
_x1   [x,1,$]||   ⊢         match x
__1     [1,$]||   ⊢         match 1
___       [$]||   ⊢
___        []||
```

만약 $S \to AB$ 같이 두 개 이상의 변수로 확장되는 경우, 왼쪽이 스택 top쪽이니 먼저 가장 왼쪽 A 부터 처리하게 됨. 이는 CFG의 leftmost derivation에 대응되며, LL 파서가 이런 흐름을 따름.

- Bottom-up 방식 (스택을 오른쪽으로 눕혀 오른쪽이 top인 형태. `||`는 바닥을 표시)

```
0x1 ||[$]         ⊢         shift 0
_x1 ||[$,0]       ⊢         shift x
__1 ||[$,0,x]     ⊢         reduce by (2)
__1 ||[$,0,S]     ⊢         shift 1
___ ||[$,0,S,1]   ⊢ ⋯ ⊢     reduce by (1)  자잘한 중간 상태 2개를 거쳐서
___ ||[$,S]       ⊢
___ ||[$]         ⊢
___ ||[]
```

문법규칙 오른편에 변수가 둘 이상인 경우 bottom-up 방식 PDA 실행/계산/전이 과정 예시

(1) $S \to AB$
(2) $A \to ab$
(3) $B \to cde$

```
abcde ||[$]           ⊢        shift a
_bcde ||[$,a]         ⊢        shift b
__cde ||[$,a,b]       ⊢ ⋯ ⊢    reduce by (2)
__cde ||[$,A]         ⊢        shift c
___de ||[$,A,c]       ⊢        shift d
____e ||[$,A,c,d]     ⊢        shift e
_____ ||[$,A,c,d,e]   ⊢ ⋯ ⊢    reduce by (3)
_____ ||[$,A,B]       ⊢ ⋯ ⊢    reduce by (1)
_____ ||[$,S]         ⊢
_____ ||[$]           ⊢
_____ ||[]
```

PDA의 실행 시간 순서상으로는 왼쪽의 A가 먼저 처리되지만, bottom-up 방식은 derivation을 시간을 거꾸로 되감아 돌리듯 역순으로 진행시키고 있음에 유의!!! 결과적으로 전체 derivation을 표준적인 방향으로 작성하면 $S \Rightarrow AB \Rightarrow Acde \Rightarrow abcde$ 이므로 derivation의 정방향 진행 순서상으로는 오른쪽의 B부터 처리되는 rightmost derivation에 해당하며, LR 파서가 이런 방식으로 작동. 우리 교재에서는 derivation의 역방향을 reduction이라는 이름을 붙여서 다루고 있음. 그러니까 $abcde \rightarrowtail Acde \rightarrowtail AB \rightarrowtail S$ 이걸 leftmost reduction으로 부름.

Deterministic CFG (DCFG)와 Deterministic PDA (DPDA)

용어 정의

DCFL DPDA로 인식/판별 가능한 언어

DPDA δ가 비결정적으로 한 단계 전이 후 여러 상황에 대응 가능한 일반적인 관계(relation) 형태가 아니라 다음 상태와 스택 변화가 유일하게 하나로 결정되도록 δ가 전이 함수(function)

DCFG CFG가 표현하는 언어에 속하는 모든 문자열에 대한 leftmost reduction, 즉, 시작 변수로 축소에 성공하는 leftmost reduction 과정이 항상 결정적인 CFG. (언어에 속하는 문자열로 시작하는 가장 왼쪽부터 축소 과정의 매 단계에서 적용 가능한 규칙이 항상 하나뿐이라는 말) 정의는 forced handle이라는 조금 더 세부적인 개념을 사용해서 정의하는데 대략의 느낌은 이렇다.

DCFG의 정의 자체만로는 결정적 문법분석 알고리즘의 존재를 당연하게 확신하기는 곤란. 왜냐하면 언어에 속하지 않는 문자열의 (결과적으로 시작 변수로 축소되지 않는) leftmost reduction까지도 항상 결정적이라는 조건은 정의에 포함되지 않기 때문. 문법분석 알고리즘은 주어진 문자열이 언어에 속하는지 미리 알고 돌리는 게 아니니까 …

그래서 큰 줄기의 스토리는 DPDA랑 DCFG가 쌤쌤이라는 …

그러니까 DCFG의 형태인지 검사하면 문법분석 알고리즘을 결정적으로 설계 가능하다는 결론

DCFG 문법의 형태/성질 기준 문법분석 알고리즘 분류

크게 LL계열과 LR계열로 분류하는데
- 공통적인 첫 글자 L은 왼쪽에서부터 오른쪽으로 처리한다는 문자열 처리 방향
- 둘째 글자는 문자열의 분석 과정을 derivation으로 나타냈을 때 Leftmost인지 Rightmost인지

아래에서 k는 2이상의 정해진 자연수

LL(0), LL(1), LL(k) 문법
- Leftmost derivation으로 진행
- top-down 하향식, recursive decent 재귀하강 … 그런 이름과 용어를 사용하는 파서로 주로 구현
- 0, 1, k개의 미리보기(lookahead) 심볼을 사용하여 결정적 파싱 가능
  $\{\mathcal{L}(G) \mid G\text{는 LL}(0)\} \subset \{\mathcal{L}(G) \mid G\text{는 LL}(1)\} \subset \{\mathcal{L}(G) \mid G\text{는 LL}(\text{k})\} \subset \text{전체 DCFL}$

LR(0), LR(1), LR(k) 문법
- Rightmost derivation으로 진행 (DPDA 실행 시간 순서로는 leftmost reduction)
- bottom-up 상향식, table driven, … shift reduce … 그런 이름과 용어를 사용하는 파서로 주로 구현
- 0, 1, k개의 미리보기(lookahead) 심볼을 사용하여 결정적 파싱 가능
  $\{\mathcal{L}(G) \mid G\text{는 LR}(0)\} \subset \{\mathcal{L}(G) \mid G\text{는 LR}(1)\} = \{\mathcal{L}(G) \mid G\text{는 LR}(\text{k})\} = \text{전체 DCFL}$

그러니까 LR(1) 문법으로 모든 DCFL을 다 나타낼 수 있음.

주교재 3 처치-튜링 명제/논제/테제/가설

The Church-Turing Thesis는 “계산 가능”하다는 것이 무슨 의미인지 …
즉, “계산”이란 무엇인지에 대한 정의를 제시 (현명한 사람의 직관, 번뜩이는 아이디어, 경험적 지혜, 꿈에서 받은 계시 등에 전혀 의존하지 않고) 고정된 규칙에 따라 누구나, 심지어 영혼 없는 기계에 시키기만 하면 자동으로 알아서 진행 가능한 기계적인 “계산”이란 무엇인지 설명하려는 다양한 모델들 중 대표적인 두 가지

- Alonzo Church의 람다계산법(λ-calculus)
- Alan Turing의 튜링 기계(Turing machine, 줄여서 TM)

이 두 모델은 (그리고 몇몇 다른 모델들도) 서로 다른 사람들이 서로 다른 방식으로 시도했지만 이들이 제시한 계산 가능성의 범위는 동일하다 … 모두 다 쌤쌤이라는 것이 밝혀짐.

그래서 이렇게 여러 훌륭한 천재적인 학자들이 독립적으로 시도한 결과가 동일하다면 아마도 기계적 “계산”이라는 현상의 본질을 정확히 인류가 파악하게 된 것이라고 잠정적으로 받아들이고 이것을 벗어나는 “계산”이라는 개념은 없을 것이라고 판단하고 있는 …
이러한 명제/논제/테제/가설(thesis)을 “Church-Turing thesis”라고 함.

흥미로운 역사적 사실

- TM이 가장 먼저 나오고 그 다음 FA와 PDA 순으로 등장
- Church는 Turing의 박사 지도교수였으며 람다계산법이 TM보먼 먼저 등장
  (당시에는 인터넷이 없었으므로 미국과 영국에서 각자가 독자적으로 고안한 모델)

3.1 Turing Machines

PDA로 인식/처리 못하는, 즉 CFL이 아닌 대표적인 언어들

- $a^k b^k c^k$ 처럼 세 부분의 길이(또는 반복 횟수)가 같은 걸 확인해야 하는 언어
- $w\#w$ 처럼 정확히 같은 문자열이 정확히 같은 순서로 반복되는지 확인 (참고로, $w\#w^R$은 CFL)